

NaN (Need a Name)

1 General

1.1 Plantilla

```
1 #define FIN ios::sync_with_stdio(0);cin.tie(0);cout
   .tie(0)
```

1.2 Compilar

```
1 g++ -std=c++17 -DLOCAL -g -O2 -Wconversion -Wshadow
   -Wall -Wextra -D_GLIBCXX_DEBUG -c %f
2 g++ -std=c++17 -DLOCAL -g -O2 -Wconversion -Wshadow
   -Wall -Wextra -D_GLIBCXX_DEBUG -o %e %f
3
4 code ~/.bashrc # archivo para crear alias
5
6 compile() {
7     g++ "$1.cpp" -o "$1" 2>&1|grep "error:"
8 } # Crea ejecutable con mismo nombre que el archivo
   .cpp
9
10 source ~/.bashrc # resetear el terminal para
   aplicar cambios
11 ulimit -s 1048576 # aumentar el stack a 1Gb (Ubuntu
   )
```

2 Data Structures

2.1 Compresion De Coordenadas

```
1 struct compresion {
2     vector<int> todos;
3     compresion(vector<int> v) {
4         todos = v; sort(all(todos));
5         todos.erase(unique(all(todos)),todos.end())
6         ;
7     }
8     int obtener(int x) { return (int)(lower_bound(
   all(todos),x)-todos.begin()); }
9 };
10
```

2.2 Dsu

```
1 struct DSU {
2     vi link, sz;
3     DSU(int tam) {
4         link.resize(tam+5), sz.resize(tam+5);
5         forn(i, tam+5) link[i] = i, sz[i] = 1;
6     }
7     ll find(ll x){ return link[x] = (link[x] == x ?
   x : find(link[x])); }
8     bool same(ll a, ll b) { return find(a) == find(
   b); }
9     void join(ll a, ll b) {
10         a = find(a), b = find(b);
11         if(a == b) return;
12         if(sz[a] < sz[b]) swap(a,b);
13         sz[a] += sz[b];
14         link[b] = a;
15     }
16 };
17
```

2.3 Implicit Treap

```
1 typedef struct item *pitem;
2 struct item {
3     ll key, prior, cont, mini, sum;
4     bool rev; // (parameters for lazy prop)
5     pitem l, r;
6     item(ll key):
7         key(key),prior(rand()),cont(1),l(NULL),r(
   NULL),rev(0) {}
8 };
9 void push(pitem it) { //Lazy for reverse array
10     if(it && it->rev) {
11         swap(it->l,it->r);
12         if(it->l)it->l->rev ^= true;
13         if(it->r)it->r->rev ^= true;
14         it->rev = false;
15     }
16 }
17 int cont(pitem it) {return it ? it->cont : 0;}
18 ll mini(pitem it) {return it ? it->mini : 1<<30;}
19 ll sum(pitem it) {return it ? it->sum : 0LL;}
20 void upd_cont(pitem it){
21     if(it) {
22         it->cont = cont(it->l) + cont(it->r) + 1;
23         it->mini = min(it->key,min(mini(it->l),mini
   (it->r)));
24         it->sum = it->key + sum(it->l) + sum(it->r)
   ;
25     }
26 }
27 void merge(pitem &t, pitem l, pitem r) { //Merge l
   and r, new root t
28     push(l); push(r);
29     if(!l || !r) t = l ? l : r;
30     else if(l->prior > r->prior) merge(l->r,l->r,r)
   , t=l;
31     else merge(r->l,l,r->l), t = r;
32     upd_cont(t);
33 }
34 void split(pitem t, pitem& l, pitem& r, int sz){ //
   sz:desired size of l
35     if(!t) { l = r = 0; return;}
36     push(t);
37     if(sz <= cont(t->l)) split(t->l,l,t->l,sz), r =
   t;
38     else split(t->r,t->r,r,sz-1-cont(t->l)), l = t;
39     upd_cont(t);
40 }
41 int find_min(pitem t) { //Devuelve la posicion del
   minimo
42     push(t);
43     if(t->mini == t->key) return cont(t->l);
44     else if(t->l && t->l->mini == t->mini) return
   find_min(t->l);
45     else return cont(t->l)+1+find_min(t->r);
46 }
47 void set_treap(pitem &root, int p, ll val) {
48     pitem aux1 = NULL, aux2 = NULL;
49     split(root,root,aux2,p+1); split(root,root,aux1
   ,p);
50     aux1->key = val;
51     merge(root,root,aux1); merge(root,root,aux2);
52 }
53 ll get_sum(pitem &root, int l, int r) {
54     pitem aux1 = NULL, aux2 = NULL;
55     split(root,root,aux2,r+1); split(root,root,aux1
```

```

    ,l);
56     ll ans = aux1->mini; // aux1->mini for minimum
57     merge(root,root,aux1); merge(root,root,aux2);
58     return ans;
59 }
60 void DFS(pitem t) { //Good for debug
61     DBG(t->key);
62     if(t->l != NULL) DBG(t->l->key); if(t->r !=
        NULL) DBG(t->r->key);
63     RAYA;
64     if(t->l != NULL) DFS(t->l); if(t->r != NULL)
        DFS(t->r);
65 }

```

2.4 Indexed Set

```

1 using namespace __gnu_pbds;
2 typedef tree<ll,null_type,less<ll>,rb_tree_tag,
    tree_order_statistics_node_update> indexed_set;
3 indexed_set s; s.insert(2); s.insert(3); s.insert
    (5);
4 auto x=s.find_by_order(2); // retorna el valor en
    la posicion 2 (5)
5 int pos=int(s.order_of_key(3)); // retorna la
    posicion que se encuentra el 3 (1)

```

2.5 Lazy-Propagation

```

1 typedef long long tipo;
2 const int NEUT = 0; // REMINDER !!!
3 struct node {
4     tipo ans, l, r, lazy = 0;
5     bool upd = false;
6     node() { ans = lazy = 0; upd = false; l = r = -1;
7         } // REMINDER !!! SET NEUT
8     node(tipo val, int pos) : ans(val), l(pos), r(pos)
9         {} // Set node
10    void set_lazy(tipo x) { lazy += x; upd = true; }
11 };
12 struct segtree_lazy {
13     #define l(x) int(x<<1)
14     #define r(x) int(x<<1|1)
15     vector<node> t; int tam;
16     node op(node a, node b) {
17         node aux; aux.ans = a.ans + b.ans; //Operacion
18         de query
19         aux.l = a.l; aux.r = b.r;
20         return aux;
21     }
22     void node_update(node &cur) {
23         cur.ans += cur.lazy * (cur.r-cur.l+1); //
24         Operacion update
25     }
26     void reset_lazy(node &cur) {
27         cur.lazy = 0; cur.upd = false; //Poner el
28         neutro del update
29     }
30     void push(int p) {
31         node &cur = t[p];
32         if(cur.upd == true) {
33             node_update(cur);
34             if(cur.l < cur.r) {
35                 t[l(p)].set_lazy(cur.lazy);
36                 t[r(p)].set_lazy(cur.lazy);
37             }
38             reset_lazy(cur);
39         }
40     }
41 }

```

```

34     }
35 }
36 node query(int l, int r, int p = 1) {
37     push(p); node &cur = t[p];
38     if(l > cur.r || r < cur.l) return node(); //
39     Return NEUT
40     if(l <= cur.l && cur.r <= r) return cur;
41     return op(query(l,r,l(p)),query(l,r,r(p)));
42 }
43 void update(int l, int r, tipo val, int p = 1) {
44     // root at p = 1
45     push(p); node &cur = t[p];
46     if(l > cur.r || r < cur.l) return;
47     if(l <= cur.l && cur.r <= r) {
48         cur.set_lazy(val); push(p); return;
49     }
50     update(l, r, val, l(p)); update(l, r, val, r(p))
51     );
52     cur = op(t[l(p)], t[r(p)]);
53 }
54 void build(vector<tipo> v, int n) { // iterative
55     build
56     tam = sizeof(int) * 8 - __builtin_clz(n); tam =
57     1<<tam;
58     t.resize(2*tam); v.resize(tam);
59     forn(i,tam) t[tam+i] = node(v[i],i);
60     for(int i = tam - 1; i > 0; i--) t[i] = op(t[l(
61         i)],t[r(i)]);
62 }
63 }
64 };
65 }

```

2.6 Mo Dquery

```

1 struct query{
2     int l, r, ind;
3     bool operator <(query q) const {
4         return r < q.r;
5     }
6 };
7 struct mo {
8     const int block = 448; // sqrt(MAXN)
9     vector<vector<query>> q;
10    vector<ll> ans, cont;
11    mo(){
12        ans.resize(MAXN), cont.resize(MAXN);
13        q.resize(MAXN/block+5);
14    }
15    void read_queries(int m) { // m --> number of
16        queries
17        ans.resize(m,0);
18        forn(i, m) {
19            int l, r; cin >> l >> r;
20            l--; r--; // For 0-indexed
21            q[l / block].pb({l,r,(int)i});
22        }
23    }
24    void add(ll &sum, ll val) {
25        if(cont[val] == 0) sum++;
26        cont[val]++;
27    }
28    void erase(ll &sum, ll val) {
29        if(cont[val] == 1) sum--;
30        cont[val]--;
31    }
32    void get_mo(vector<ll> &v) {
33    }
34 }

```

```

32     ll sum = 0;
33     forn(i,q.size()) {
34         sort(all(q[i]));
35         int l, r; l = r = block * i;
36         for(query u : q[i]) { // Solve for [l,r]
37             while(r <= u.r) add(sum,v[r]), r++;
38             while(l <= u.l) erase(sum,v[l]), l++;
39             while(l > u.l) l--, add(sum,v[l]);
40             ans[u.ind] = sum;
41         }
42         while(l < r) erase(sum,v[l]), l++;
43     }
44 }
45 };

```

2.7 Persistent

```

1 struct Vertex {
2     Vertex *l, *r;
3     ll sum;
4     Vertex(ll val) : l(nullptr), r(nullptr), sum(
5         val) {}
6     Vertex(Vertex *l, Vertex *r) : l(l), r(r), sum
7         (0) {
8         if (l) sum += l->sum;
9         if (r) sum += r->sum;
10    }
11 };
12 Vertex* build(ll a[], ll tl, ll tr) {
13     if (tl == tr)
14         return new Vertex(a[tl]);
15     ll tm = (tl + tr) / 2;
16     return new Vertex(build(a, tl, tm), build(a, tm
17         +1, tr));
18 }
19 ll get_sum(Vertex* v, ll tl, ll tr, ll l, ll r) {
20     if (l > r)
21         return 0;
22     if (l == tl && tr == r)
23         return v->sum;
24     ll tm = (tl + tr) / 2;
25     return get_sum(v->l, tl, tm, l, min(r, tm))
26         + get_sum(v->r, tm+1, tr, max(l, tm+1), r)
27         ;
28 }
29 Vertex* update(Vertex* v, ll tl, ll tr, ll pos, ll
30     new_val) {
31     if (tl == tr)
32         return new Vertex(new_val);
33     ll tm = (tl + tr) / 2;
34     if (pos <= tm)
35         return new Vertex(update(v->l, tl, tm, pos,
36             new_val), v->r);
37     else
38         return new Vertex(v->l, update(v->r, tm+1,
39             tr, pos, new_val));
40 }
41 vector <Vertex*> estado;

```

2.8 Segtree-Iterative

```

1 typedef long long tipo;
2 struct segtree {
3     vector <tipo> t; int tam;

```

```

4     tipo NEUT = 0;
5     tipo op(tipo a, tipo b){ return a+b; }
6     void build(vector<tipo>v, int n) {
7         tam = sizeof(int) * 8 - __builtin_clz(n); tam =
8             1<< tam;
9         t.resize(2*tam,NEUT); forn(i, n) t[tam+i] = v[i]
10            ];
11         for(int i = tam - 1; i > 0; i--) t[i] = op(t[i
12             <<1], t[i<<1+1]);
13     }
14     void update(int p, tipo value) {
15         for (t[p += tam] = value; p > 1; p >>= 1) t[p
16             >>1] = op(t[p], t[p^1]);
17     }
18     tipo query(int l, int r) {
19         tipo res = NEUT;
20         for (l += tam, r += tam; l <= r; l >>= 1, r >>=
21             1) {
22             if (l&1) res = op(res, t[l++]);
23             if (!(r&1)) res = op(res, t[r--]);
24         }
25         return res;
26     }
27 }
28 };

```

2.9 Segtree 2D

```

1 struct st2d{ // 0 indexado, [l, r]
2     // PARA HACER BUILD HAY QUE HACER LOS N*N updates
3     int n; vector<vi> st; ll NEUT=0;
4     ll op(ll a, ll b){return a+b;}
5     st2d(int _n): n(_n) { st.resize(2*n+5, vi(2*n+5,
6         0)); }
7     void upd(int x, int y, ll v){
8         st[x+n][y+n]=v;
9         for(int j=y+n;j>1;j>>=1)st[x+n][j>>1]=op(st[x+n
10             ][j],st[x+n][j^1]);
11         for(int i=x+n;i>1;i>>=1)for(int j=y+n;j;j>>=1)
12             st[i>>1][j]=op(st[i][j],st[i^1][j]);
13     }
14     ll query(int x0, int x1, int y0, int y1){
15         ll r=NEUT; x1++, y1++;
16         for(int i0=x0+n,i1=x1+n;i0<i1;i0>>=1,i1>>=1){
17             int t[4],q=0;
18             if(i0&1)t[q++] = i0++;
19             if(i1&1)t[q++] = --i1;
20             forn(k,q) for(int j0=y0+n,j1=y1+n;j0<j1;j0
21                 >>=1,j1>>=1){
22                 if(j0&1)r=op(r,st[t[k]][j0++]);
23                 if(j1&1)r=op(r,st[t[k]][--j1]);
24             }
25         }
26         return r;
27     }
28 }
29 };

```

3 Dp

3.1 Cht Dynamic

```

1 typedef ll tc;
2 const tc is_query=-(1LL<<62);
3 struct Line {
4     tc m,b;
5     mutable multiset<Line>::iterator it,end;

```

```

6     const Line* succ(multiset<Line>::iterator it)
7         const {
8             return (++it==end? NULL : &*it);}
9     bool operator<(const Line& rhs) const {
10         if(rhs.b!=is_query)return m<rhs.m;
11         const Line *s=succ(it);
12         if(!s)return 0;
13         return b-s->b<(s->m-m)*rhs.m;
14     }
15 }; // Para minimo, cambiar el signo de m y b.
16 struct HullDynamic : public multiset<Line> {
17     bool bad(iterator y){ // for maximum
18         iterator z=next(y);
19         if(y==begin()){
20             if(z==end())return false;
21             return y->m==z->m&&y->b<=z->b;
22         }
23         iterator x=prev(y);
24         if(z==end())return y->m==x->m&&y->b<=x->b;
25         return (x->b-y->b)*(z->m-y->m)>=(y->b-z->b)
26             *(y->m-x->m);
27     }
28     iterator next(iterator y){return ++y;}
29     iterator prev(iterator y){return --y;}
30     void add(tc m, tc b){
31         // m *= -1; b *= -1;
32         iterator y=insert((Line){m,b});
33         y->it=y->end=end();
34         if(bad(y)){erase(y);return;}
35         while(next(y)!=end()&&bad(next(y)))erase(
36             next(y));
37         while(y!=begin()&&bad(prev(y)))erase(prev(y)
38             ));
39     }
40     tc eval(tc x){
41         Line l=*lower_bound((Line){x,is_query});
42         return l.m*x+l.b; // -1*(l.m*x + l.b)
43     }
44 };

```

3.2 Divide And Conquer Dp Opt

```

1 ll costo[MAXN][MAXN];
2 vector<ll> last(MAXN), dp(MAXN);
3 void calc_costo(int n, vector<ll> &v) {}
4 void compute(int l, int r, int optl, int optr) {
5     if(l > r) return;
6     int med = (l+r)/2;
7     pair<ll,ll> best = {INF,-1};
8     for(int p = optl; p <= min(med,optr); p++) {
9         best = min(best,{last[p] + costo[p+1][med],
10             p});
11     }
12     dp[med] = best.first;
13     int opt = best.second;
14     compute(l,med-1,optl,opt);
15     compute(med+1,r,opt,optr);
16 }
17 ll solve_dac(int n, int k) {
18     for(int i = 0; i < n; i++) last[i] = costo[0][i];
19     for(int i = 2; i <= k; i++) {
20         fill(all(dp),INF);
21         compute(0,n-1,0,n-1);
22         last = dp;

```

```

22     }
23     return dp[n-1];
24 }

```

3.3 Knapsack Weighted

```

1 vector<bool> knapsack_weighted(vector<int> v, int
2     n) {
3     int sum = accumulate(all(v),0LL);
4     vector<bool> dp(sum+1,false);
5     dp[0] = true;
6     sort(all(v));
7     vector<pair<int,int>> num; //(num,#count)
8     int count = 1, last = v[0];
9     forr(i,1,n) {
10         if(v[i] == last) count++;
11         else {
12             num.pb({last,count});
13             last = v[i];
14             count = 1;
15         }
16     }
17     num.pb({last,count});
18     for(auto [x,c] : num) {
19         vector<bool> next_dp = dp;
20         forn(i,x) {
21             int can = 0;
22             for(int j = i; j <= sum; j += x) {
23                 next_dp[j] = next_dp[j] | (can>0);
24                 if(dp[j]) can++;
25                 if(j - c*x >= 0 and dp[j-c*x]) can--;
26             }
27             dp = next_dp;
28         }
29     }
30     return dp;

```

3.4 Knuth Division Dp Opt

```

1 const ll INF = (1ll)(1e18+10);
2 const int MAXN = 5005;
3 vector<ll> prefix;
4 // Sea Pos(i,j) la posicion del corte que optimiza
5 // la dp, intervalo [i,j]
6 // Se puede aplicar optimizacion de Knuth si se
7 // cumple que:
8 // Pos(i,j-1) <= pos(i,j) <= pos(i+1,j)
9 // Objetivo: llevar un array a elementos
10 // individuales mediante cortes.
11 ll suma(int i, int j) {
12     return prefix[j+1] - prefix[i];
13 } // Suma de los elementos del inter
14 ll solve_Knuth(int n) {
15     vector<ll> pos(n+1);
16     vector<vector<ll>> dp(MAXN,vector<ll>(MAXN,
17         INF));
18     for(int i = 0; i < n; i++) dp[i][i] = 0, pos[i]
19         = i;
20     for(int len = 1; len < n; len++) {
21         vector<ll> aux(n+1,0);
22         for(int i = 0; i < n - len; i++) {
23             for(int k = pos[i]; k <= pos[i+1]; k++)
24                 {
25                     ll cost = suma(i,i+len) + dp[i][k]
26                         + dp[k+1][i+len];

```

```

20         if(cost < dp[i][i+len]) {
21             dp[i][i+len] = cost; aux[i] = k
                ;
22         }
23     }
24 }
25 pos = aux;
26 }
27 return dp[0][n-1];
28 }

```

3.5 Lis

```

1 ll lis(vi &a, int strict = 0){
2     vi temp; temp.pb(a[0]);
3     forr(i, 1, SZ(a)){
4         ll x = a[i];
5         if(x >= temp.back()+strict) temp.pb(x);
6         else { auto it = upper_bound(all(temp), x-
            strict); *it = x; } }
7     return SZ(temp); }

```

4 Geometry

4.1 Angular Sort

```

1 // Angular sort
2 punto origin = {OLL,OLL};
3 bool AngularSort(const punto &A, const punto &B) {
4     bool aorig = (A < origin); // semiplano izquierdo
        o derecho
5     bool borig = (B < origin); // semiplano izquierdo
        o derecho
6     if (aorig != borig) { return aorig > borig; } //
        primero sem izq
7     return (A~B) > 0; // desempate por producto
        vectorial si mismo sem
8 }

```

4.2 Calipers

```

1 // Devuelve la distancia entre los puntos mas
    lejanos en O(N)
2 // Cuando se usa con nuestra convex_hull, hacer
    reverse porque vienen CW
3 ld callipers(vector<punto> p, int n){
4     ld r=0; // prereq: Convex, ccw, NO
        COLLINEAR POINTS
5     for(int i=0,j=n<2?0:1;i<j;++i){
6         for(;;j=(j+1)%n){
7             r=max(r, (p[i]-p[j]).mod());
8             if((p[(i+1)%n]-p[i])^(p[(j+1)%n]-p[j])
                <=EPS)break;
9         }
10    }
11    return r;
12 }

```

4.3 Closest Points Monogon

```

1 const ll INF = (ll)(1e15+10); // (1e18+10)
2 template<typename T>
3 using minpq = priority_queue<T, vector<T>, greater<
    T>>;
4 struct punto {
5     ll x, y;

```

```

6     punto (ll x = 0, ll y = 0) : x(x), y(y) {}
7     punto operator -(const punto &p) const {
8         return punto(x - p.x, y - p.y); }
9     ll len2() const { return x * x + y * y; }
10    bool const operator <(const punto &p) const {
11        return (x != p.x) ? x < p.x : y < p.y;
12    }
13 };
14 istream& operator>>(istream &is, punto &p) {
15     return is >> p.x >> p.y;
16 }
17 ll closest(vector<punto> &p, int l, int r) { //Init
    l = 0 and r = p.size()-1
18     if(l == r) return INF;
19     int m = (l + r) / 2;
20     ll mid = p[m].x;
21     ll d = min(closest(p, l, m), closest(p, m + 1,
        r));
22     vector<punto> A, B, C;
23     forr(i, l, m + 1) {
24         ll r = mid - p[i].x;
25         if(r * r <= d) A.push_back(p[i]);
26     }
27     forr(i, m + 1, r + 1) {
28         ll r = mid - p[i].x;
29         if(r * r <= d) B.push_back(p[i]);
30     }
31     auto cmpy = [&](punto a, punto b) { return a.y
        < b.y; };
32     merge(all(A), all(B), back_inserter(C), cmpy);
33     forr(i, 0, C.size()) {
34         int j = i + 1;
35         while(j < C.size() && (C[i] - C[j]).y * (C[
            i] - C[j]).y <= d) {
36             d = min(d, (C[i] - C[j]).len2());
37             j++;
38         }
39     }
40     inplace_merge(p.begin() + l, p.begin() + m + 1,
        p.begin() + r + 1, cmpy);
41     return d;
42 }
43 void sorting(vector <punto> &p) { sort(all(p)); }
    // Don't Forget

```

4.4 Convex Hull

```

1 const ld EPS = 1e-10;
2 void convex_hull(vector<punto> &a) {
3     if(SZ(a) == 1) return;
4     sort(all(a));
5     punto p1 = a[0];
6     vector<punto> up, down;
7     up.pb(p1); down.pb(p1);
8     forr(i,1,SZ(a)) {
9         int n = SZ(up), m = SZ(down);
10        while(n > 1 && ((up[n-1]-up[n-2])^(a[i]-up[
            n-1])) >= -EPS) {
11            up.pop_back(); n--;
12        } up.pb(a[i]);
13        while(m > 1 && ((down[m-1]-down[m-2])^(a[i
            ]-down[m-1])) <= EPS) {
14            down.pop_back(); m--;
15        } down.pb(a[i]);
16    } // Cambiar EPS a 0 para mejor precision en

```

```

    enteros.
17  a.clear();
18  for(punto u : up) a.pb(u);
19  for(int i = SZ(down)-2; i > 0; i--) a.pb(down[i
    ]);
20 }

```

4.5 Formulas

```

1 // Shoelace formula
2 tipo area(vector <punto> &v) {
3     tipo ans = 0.0; int n = v.size();
4     forn(i,n) ans += v[i] ^ v[(i+1)%n];
5     return fabs(ans/2.0);
6 }
7 tipo find_alpha(recta r, punto p) {
8     return r.v.x != 0 ? (p.x-r.p.x)/r.v.x : (p.y-r
        .p.y)/r.v.y;
9 }

```

4.6 Intersection All

```

1 struct recta { // Puede usarse para segmentos ([p,p
    +v] o alpha = [0,1])
2     punto v, p; // v -> director, p -> punto por
        donde pasa
3     recta(punto p1, punto p2) { v = (p2-p1); p = p1
        ;}
4     recta() {}
5     recta(tipo A, tipo B, tipo C) { // Transform Ax
        + By + C = 0
6         v = {-B,A}; A != 0 ? p = {-C / A,0} : p =
            {0,-C / B};
7     }
8     bool is_in(punto q){return fabs((q.x-p.x)*v.y -
        (q.y-p.y)*v.x) < EPS;}
9     punto eval(double x) {return x * v + p;}
10 };
11 bool inter_recta(recta &r1, recta &r2, punto &ans)
    {
12     // Retorna false si son paralelas, sino guarda
        el punto en ans
13     if(abs(r1.v ^ r2.v) < EPS) return false;
14     tipo alpha = tipo((r2.p - r1.p)^r2.v) / tipo(r1
        .v^r2.v);
15     ans = r1.p + alpha * r1.v;
16     return true;
17 }
18 bool inter_seg(recta &r1, recta &r2, punto &ans) {
19     if(r1.p == r2.p || r1.p == r2.p+r2.v) {ans = r1
        .p; return true;}
20     if(r1.p+r1.v == r2.p || r1.p+r1.v == r2.p+r2.v)
        {
21         ans = r1.p+r1.v; return true; } //Casos que
            coincidan extremos
22     if(abs(r1.v ^ r2.v) < EPS) return false; // son
        paralelos
23     tipo alpha = tipo((r2.p - r1.p)^r2.v) / tipo(r1
        .v^r2.v);
24     tipo beta = tipo((r1.p - r2.p)^r1.v) / tipo(r2.
        v^r1.v);
25     if(alpha < -EPS || beta < -EPS) return false;
26     if(alpha > 1.0+EPS || beta > 1.0+EPS) return
        false;
27     ans = r1.p + alpha * r1.v; return true;
28 }

```

```

29 struct circ { punto c; tipo r; };
30 tipo dist_point_line(punto &p, recta &r) {
31     punto p1 = r.p, p2 = r.p + r.v;
32     return fabs((p1-p)^2)/r.v.mod();
33 }
34 punto project(punto a, punto b) { //Proyeccion de b
    sobre a
35     return ((a*b)/a.mod2()) * a;
36 }
37 vector<punto> inter_circ_line(recta r, circ c) {
38     vector <punto> ans; tipo dist = dist_point_line
        (c.c,r);
39     if(dist > c.r+EPS) return ans;
40     (c.c-r.p) * r.v != 0 ? r.p = r.p : r.p = r.p +
        r.v;
41     punto aux = c.c - r.p, dir = project(r.v,aux);
42     if(fabs(dist-c.r) <= EPS) {ans.pb(r.p + dir);
        return ans;}
43     tipo factor = sqrt(c.r*c.r - dist*dist)/dir.mod
        ();
44     ans.pb(r.p + dir + factor * dir); ans.pb(r.p +
        dir - factor * dir);
45     return ans;
46 }
47 tipo intersection_area(circ a, circ b) {
48     punto aux = (b.c - a.c); tipo dist=aux.mod(),
        dist2=aux.mod2();
49     if(a.r + b.r - dist < -EPS) return 0;
50     if(fabs(a.r - b.r) - dist > -EPS) {
51         return min(a.r, b.r) * min(a.r, b.r) * 2*
            acosl(0); }
52     tipo alpha = acosl((dist2 + a.r*a.r - b.r*b.r)
        / (2 * dist * a.r));
53     tipo beta = acosl((dist2 + b.r*b.r - a.r*a.r) /
        (2 * dist * b.r));
54     tipo ans1 = (alpha - sinl(alpha+alpha)*0.5) * a
        .r * a.r;
55     tipo ans2 = (beta - sinl(beta+beta)*0.5) * b.r
        * b.r;
56     return ans1 + ans2;
57 }
58 vector<punto> inter_circ_circ(circ a, circ b) {
59     vector <punto> ans;
60     if (a.c == b.c) {
61         return abs(a.r-b.r) <= EPS ? vector<punto>{
            a.c,a.c,a.c} : ans;
62     }
63     b.c = b.c - a.c; punto aux = a.c; a.c = a.c - a
        .c;
64     recta r(-2*b.c.x, -2*b.c.y, a.r*a.r - b.r*b.r +
        b.c.x*b.c.x + b.c.y*b.c.y);
65     ans = inter_circ_line(r,a);
66     forn(i,ans.size()) ans[i] = ans[i] + aux;
        return ans;
67 }
68 void tangents(punto c, double r1, double r2, vector
    <recta> & ans) {
69     double r = r2 - r1;
70     double z = c.xc.x + c.yc.y;
71     double d = z - rr;
72     if (d < -EPS) return;
73     d = sqrt(abs(d));
74     recta l((c.x r + c.y * d) / z, (c.y * r - c.x *
        d) / z, r1);
75     ans.push_back(l);

```

```

76 }
77 vector<recta> tangents(circ a, circ b) {
78     vector<recta> ans;
79     for (int i=-1; i<=1; i+=2) for (int j=-1; j<=1;
80         j+=2) tangents (b.c-a.c, a.ri, b.rj, ans);
81     forn(i,SZ(ans)) ans[i].p =ans[i].p + a.c;
82     return ans;
83 }

```

4.7 Minimum Enclosing Circle

```

1 punto getCircCenter(punto a, punto b) {
2     ld x = a.mod2(), y = b.mod2(), z = a^b;
3     return {(b.y * x - a.y * y) / (2 * z), (a.x * y
4         - b.x * x) / (2 * z) };
5 }
6 circ circleFrom(const punto &a, const punto &b,
7     const punto &c) {
8     punto o = getCircCenter(b-a,c-a);
9     return {o+a, o.mod()};
10 }
11 circ spanning_circle(vector<punto> &T) {
12     int n = T.size();
13     shuffle(all(T),rng);
14     circ C={0,0}, -1};
15     forn(i,n) if (dist(C.c,T[i])>C.r+EPS) {
16         C = {T[i], 0};
17         forn(j,i) if (dist(C.c,T[j])>C.r+EPS) {
18             C = {0.5*(T[i] + T[j]), dist(T[i],T[j])
19                 /2};
20             forn(k,j) if (dist(C.c,T[k])>C.r+EPS) {
21                 C = circleFrom(T[i], T[j], T[k]);
22             }
23         }
24     }
25     return C;
26 }

```

4.8 Point In Poly

```

1 bool point_in_poly(vector <punto> &v, punto p) { //
2     O(n) for non-convex
3     unsigned i, j, mi, mj, c = 0;
4     for(i = 0, j = v.size()-1; i < v.size(); j = i
5         ++){
6         if((v[i].y <= p.y && p.y < v[j].y) || (v[j]
7             .y <= p.y && p.y < v[i].y)) {
8             mi = i; mj = j; if(v[mi].y > v[mj].y)
9                 swap(mi,mj);
10             if(((p-v[mi]) ^ (v[mj]-v[mi])) < 0 ) c
11                 ^= 1;
12         }
13     }
14     return c;
15 }
16 bool point_in_poly_convex(vector <punto> &v, punto
17     p) { // O(log n)
18     bool ans = true;
19     if( ((v[1]-v[0]) ^ (v[2]-v[1])) >= 0) reverse(
20         all(v));
21     int a = 1, b = v.size()-1;
22     while(b-a > 1) {
23         int med = (a+b)/2;
24         if( ((v[med]-v[0]) ^ (p-v[med])) <= 0 ) a =
25             med;
26         else b = med;
27     }
28 }

```

```

19 }
20 if( ((v[a]-v[0]) ^ (p - v[a])) > 0) ans = false
21 ;
22 if( ((v[b]-v[a]) ^ (p - v[b])) > 0) ans = false
23 ;
24 if( ((v[0]-v[b]) ^ (p - v[0])) > 0) ans = false
25 ;
26 return ans;
27 }

```

4.9 Template Punto

```

1 typedef long double tipo; //Cambiar a long long
2 para operar en enteros
3 double EPS = (double)(1e-10);
4 struct punto { // Puede usarse para vectores
5     tipo x, y;
6     punto const operator -(const punto &p) const {
7         return {x-p.x,y-p.y};}
8     punto const operator +(const punto &p) const {
9         return {x+p.x,y+p.y};}
10     tipo operator *(const punto &p) const {return x
11         *p.x + y*p.y;}
12     tipo operator ^(const punto &p) const {return x
13         *p.y - y*p.x;}
14     bool operator == (const punto &p) const {
15         return (abs(x-p.x) < EPS && abs(y-p.y) <
16             EPS); // Para double
17 }
18 bool operator <(punto p) const {return x != p.x
19     ? x < p.x : y < p.y;}
20 tipo arg() {return atan2(y,x);}
21 tipo mod() {return sqrt1(x*x + y*y);}
22 tipo mod2() {return x*x + y*y;}
23 };
24 punto operator*(tipo k, const punto &p) {return {k*
25     p.x, k*p.y};}

```

5 Grafos

5.1 2Sat

```

1 struct two_sat { // 2*x es x, 2*x+1 es ~x
2     int tot;
3     vector<vector<int>> g, g_trans;
4     vb used, assignment;
5     vector<int> order, comp;
6     two_sat(int _tot): tot(_tot){
7         g.resize(tot); g_trans.resize(tot);
8     } // tot = (normales + negados)
9     void dfs1(int v) {
10         used[v] = true;
11         for(auto u: g[v]) if(!used[u]) dfs1(u);
12         order.pb(v);
13     }
14     void dfs2(int v, int cl) {
15         comp[v] = cl;
16         for(auto u: g_trans[v]) if(comp[u] == -1)
17             dfs2(u, cl);
18     }
19     bool solve() {
20         order.clear(); used.assign(tot, false);
21         comp.assign(tot, -1);
22         forn(i, tot) if(!used[i]) dfs1(i);
23         int comp_act = 0;
24     }
25 }

```

```

23     forn(i, tot){
24         auto v = order[tot-i-1];
25         if(comp[v] == -1) dfs2(v, comp_act++);
26     }
27     assignment.assign(tot/2, false);
28     forn(i, tot/2){
29         if(comp[2*i] == comp[2*i + 1]) return
30             false;
31         assignment[i] = comp[2*i] > comp[2*i
32             +1];
33     }
34     return true;
35 }
36 void add_edge(int from, int to){
37     g[from].pb(to); // from implica to
38     g_trans[to].pb(from);
39 }
40 void add_or(int v1, int v2) {
41     add_edge(v1 ^ 1, v2); // ~v1 -> v2
42     add_edge(v2 ^ 1, v1); // ~v2 -> v1
43 } // set x en true/false:
44 }; // add_or(x, x)/add_or(~x, ~x)

```

5.2 Bellman Ford

```

1 // Time: O(VE)
2 typedef long long tipo;
3 const int MAXN = 3000;
4 tipo INF = (tipo)(1e18+7);
5 struct arista {
6     int x, y; tipo w;
7 };
8 struct nodo {
9     int p; tipo d;
10 }; //f -> parent, d -> distance
11 vector<nodo> ans(MAXN);
12 vector<int> ciclo;
13 bool bFord(vector<arista> &lista, int n, int start)
14 {
15     forn(i,n) ans[i].p = i, ans[i].d = INF;
16     ans[start].d = 0; int x;
17     forn(i,n) {
18         x = -1;
19         for(arista u : lista) {
20             if(ans[u.y].d > ans[u.x].d + u.w) {
21                 ans[u.y].d = ans[u.x].d + u.w;
22                 ans[u.y].p = u.x;
23                 x = u.y;
24             }
25         }
26     }
27     if(x == -1) return false;
28     else {
29         forn(i,n) x = ans[x].p;
30         for(int v = x ;; v = ans[v].p) {
31             ciclo.push_back(v);
32             if(v == x && ciclo.size() > 1) break;
33         }
34         reverse(all(ciclo));
35         return true;
36     }
37 }

```

5.3 Biconnected

```

1 struct edge{ ll u, v, id; };

```

```

2 struct graph{
3     int n;
4     vector<vii> adj; // (to, id)
5     vector<edge> edges;
6     graph(int _n) : n(_n), adj(_n) {}
7     void add_edge(ll u, ll v){
8         ll id = SZ(edges);
9         adj[u].pb({v, id}); adj[v].pb({u, id});
10        edges.pb({u, v, id});
11    }
12    int add_node(){ adj.pb({}); return n++; }
13    vii& operator[] (ll u) { return adj[u]; }
14 };
15 struct BCC{
16     int n; graph adj;
17     vector<vi> comps;
18     vi num, low, art, stk, bridge;
19     BCC(graph &adj): n(adj.n), adj(_adj){
20         num.resize(n), low.resize(n), art.resize(n)
21         , bridge.resize(SZ(adj.edges));
22         for (ll u = 0, t; u < n; ++u) if (!num[u])
23             dfs(u, -1, t = 0);
24     }
25     void dfs(ll v, ll p, ll &t){
26         num[v] = low[v] = ++t;
27         stk.pb(v);
28         for(auto [u, id]: adj[v]) if (u != p){
29             if (!num[u]){
30                 dfs(u, v, t);
31                 low[v] = min(low[v], low[u]);
32                 if(low[u] > num[v]) bridge[id] = true;
33                 if(low[u] >= num[v]){
34                     art[v] = (num[v] > 1 || num[u]
35                         > 2);
36                     comps.pb({v});
37                     while (comps.back().back() != u
38                         )
39                         comps.back().pb(stk.back())
40                         , stk.pop_back();
41                 }
42             }
43             else low[v] = min(low[v], num[u]);
44         }
45     }
46     pair<vi, graph> build_tree(){
47         graph tree(0); vi id(n);
48         forn(v, n) if (art[v]) id[v] = tree.
49             add_node();
50         for (auto &comp : comps){
51             ll node = tree.add_node();
52             for(ll v: comp){
53                 if (!art[v]) id[v] = node;
54                 else tree.add_edge(node, id[v]);
55             }
56         }
57         return {id, tree};
58     } // Block Cut Tree
59 };

```

5.4 Blossom

```

1 /* O(nm) Los matchings estan en (i, mate[i]), con i
2    < mate[i] */
3 vi Blossom(vector<vi> &g) {
4     int n = SZ(g), timer = -1;

```

```

4   vi mate(n, -1), label(n), parent(n),
5       orig(n), aux(n, -1), q;
6   auto lca = [&](int x, int y) {
7       for (timer++; ; swap(x, y)) {
8           if (x == -1) continue;
9           if (aux[x] == timer) return x;
10          aux[x] = timer;
11          x = (mate[x] == -1 ? -1 : orig[parent[mate[x]
12              ]]);
13      }
14  };
15  auto blossom = [&](int v, int w, int a) {
16      while (orig[v] != a) {
17          parent[v] = w; w = mate[v];
18          if (label[w] == 1) label[w] = 0, q.pb(w);
19          orig[v] = orig[w] = a; v = parent[w];
20      }
21  };
22  auto augment = [&](int v) {
23      while (v != -1) {
24          int pv = parent[v], nv = mate[pv];
25          mate[v] = pv; mate[pv] = v; v = nv;
26      }
27  };
28  auto bfs = [&](int root) {
29      fill(all(label), -1);
30      iota(all(orig), 0);
31      q.clear();
32      label[root] = 0; q.pb(root);
33      forn(i, SZ(q)){
34          int v = q[i];
35          for (auto x : g[v]) {
36              if (label[x] == -1) {
37                  label[x] = 1; parent[x] = v;
38                  if (mate[x] == -1)
39                      return augment(x), 1;
40                  label[mate[x]] = 0; q.pb(mate[x]);
41              } else if (label[x] == 0 && orig[v] != orig
42                  [x]) {
43                  int a = lca(orig[v], orig[x]);
44                  blossom(x, v, a); blossom(v, x, a);
45              }
46          }
47      }
48      return 0;
49  }; //Time halves if start with any maxmatch.
50  forn(i, n) if(mate[i] == -1) bfs(i);
51  return mate;
52  }

```

5.5 Componentes Fuertemente Conexas

```

1  struct SCC {
2      int n, scc;
3      vector<vi> g, gr, ans;
4      vb visto;
5      vi order, comp_act, component;
6      SCC(vector<vi> &_g): n(SZ(_g)) {
7          g = _g;
8          gr.resize(n), visto.resize(n), component.
9              resize(n);
10         forn(v, n) for(auto u: g[v]) gr[u].pb(v);
11         find_scc();
12     }
13     void DFS1 (int v) {

```

```

13         visto[v] = true;
14         for (int u : g[v]) if(!visto[u]) DFS1(u);
15         order.pb(v);
16     }
17     void DFS2 (int v) {
18         visto[v] = true;
19         comp_act.pb(v);
20         for (int u : gr[v]) if(!visto[u]) DFS2(u);
21     }
22     void find_scc() {
23         fill(all(visto),false);
24         forn(i,n) if(!visto[i]) DFS1(i);
25         fill(all(visto),false);
26         forn(i,n) {
27             int v=order[n-i-1];
28             if(!visto[v]) {
29                 DFS2(v);
30                 ans.pb(comp_act);
31                 comp_act.clear();
32             }
33         }
34         scc = SZ(ans); // cantidad de scc's
35         forn(i, scc) for(auto v: ans[i]) component[
36             v] = i;
37     }

```

5.6 Dijkstra

```

1  struct arista {
2      int next; tipo w;
3  };
4  struct nodo {
5      tipo d, v, a;
6      bool operator<(const nodo& next) const {return
7          d > next.d;}
8  };
9  vector<nodo> Dijkstra(int start, int n, vector<
10     vector<arista>> &g) {
11     vector<nodo> ans(n);
12     vector<bool> visto(n, false);
13     priority_queue<nodo> p; p.push({0,start,-1});
14     while(!p.empty()) {
15         nodo it=p.top(); p.pop();
16         if(visto[it.v]) continue;
17         else {
18             ans[it.v] = it; visto[it.v] = true;
19             for(arista u : g[it.v]) {
20                 if(!visto[u.next]) p.push({it.d + u
21                     .w, u.next, it.v});
22             }
23         }
24     }
25     return ans;
26 }

```

5.7 Dinic

```

1  // Max Flow en  $O(V^2 E)$ . Bipartito  $O(\sqrt{V} E)$ .
2  // Matching: aristas saturadas
3  // Min cut: nodos con dist<=0 vs nodos con dist<0
4  // MVC: izq con dist<0 + der con dist>0
5  // Max Independent Set: comple de MVC (N-MVC)
6  struct Dinic{
7      int nodes,src,dst;
8      vector<int> dist,q,work;

```

```

9 struct edge { int to,rev; ll f,cap; };
10 vector<vector<edge>> g;
11 Dinic(int x):nodes(x),dist(x),q(x),work(x),g(x)
12 {}
13 void add_edge(int s, int t, ll cap){
14     g[s].pb({t,SZ(g[t]),0,cap});
15     g[t].pb({s,SZ(g[s])-1,0,0});
16 }
17 bool dinic_bfs(){
18     fill(all(dist),-1);dist[src]=0;
19     int qt=0;q[qt++]=src;
20     forn(qh, qt){
21         int u=q[qh];
22         forn(i,SZ(g[u])){
23             edge &e=g[u][i];int v=g[u][i].to;
24             if(dist[v]<0&&e.f<e.cap)dist[v]=
25                 dist[u]+1,q[qt++]=v;
26         }
27     }
28     return dist[dst]>=0;
29 }
30 ll dinic_dfs(int u, ll f){
31     if(u==dst)return f;
32     for(int &i=work[u];i<SZ(g[u]);i++){
33         edge &e=g[u][i];
34         if(e.cap<=e.f)continue;
35         int v=e.to;
36         if(dist[v]==dist[u]+1){
37             ll df=dinic_dfs(v,min(f,e.cap-e.f))
38             ;
39             if(df>0){e.f+=df;g[v][e.rev].f-=df;
40                 return df;}
41         }
42     }
43     return 0;
44 }
45 ll max_flow(int _src, int _dst){
46     src=_src;dst=_dst;
47     ll result=0;
48     while(dinic_bfs()){
49         fill(all(work),0);
50         while(ll delta=dinic_dfs(src,INF))
51             result+=delta;
52     }
53     return result;
54 }
55 };

```

5.8 Dynamic Connectivity

```

1 struct UnionFind {
2     int n,comp;
3     vi uf,si,c;
4     UnionFind(int n=0):n(n),comp(n),uf(n),si(n,1){
5         forn(i,n) uf[i]=(int)i;
6     }
7     int find(int x){return x==uf[x]?x:find(uf[x]);}
8     bool join(int x, int y){
9         if((x=find(x))==(y=find(y)))return false;
10        if(si[x]<si[y])swap(x,y);
11        si[x]+=si[y];uf[y]=x;comp--;c.pb(y);
12        return true;
13    }
14    int snap(){return SZ(c);}
15    void rollback(int snap){

```

```

16        while(SZ(c)>snap){
17            int x=c.back();c.pop_back();
18            si[uf[x]]-=si[x];uf[x]=x;comp++;
19        }
20    }
21 };
22 enum {ADD,DEL,QUERY};
23 struct Query {int type,x,y};
24 struct DynCon {
25     vector<Query> q;
26     UnionFind dsu;
27     vector<int> mt;
28     map<pair<int,int>,int> last;
29     DynCon(int n):dsu(n){
30         void add(int x, int y){
31             if(x>y)swap(x,y);
32             q.pb({ADD,x,y});mt.pb(-1);last[{x,y}]=SZ(q)
33                 -1;
34         }
35         void remove(int x, int y){
36             if(x>y)swap(x,y);
37             q.pb({DEL,x,y});
38             int pr=last[{x,y}];mt[pr]=SZ(q)-1;mt.pb(pr)
39                 ;
40         }
41         void query(int x, int y){q.pb({QUERY,x,y});mt.
42             pb(-1);}
43         void process(){ // answers all queries in order
44             if(!SZ(q)) return;
45             forn(i,SZ(q))if(q[i].type==ADD&&mt[i]<0)mt[
46                 i]=SZ(q);
47             go(0,SZ(q));
48         }
49         void go(int s, int e){
50             if(s+1==e){
51                 if(q[s].type==QUERY) // answer query
52                     using DSU q[s]
53                     assert(false); // alguna op entre q
54                     [s].x y q[s].y
55                 return;
56             }
57             int k=dsu.snap(),m=(s+e)/2;
58             for(int i=e-1;i>=m;--i)if(mt[i]>=0&&mt[i]<s
59                 )dsu.join(q[i].x,q[i].y);
60             go(s,m);dsu.rollback(k);
61             for(int i=m-1;i>=s;--i)if(mt[i]>=e)dsu.join
62                 (q[i].x,q[i].y);
63             go(m,e);dsu.rollback(k);
64         }
65     };

```

5.9 Eulerian Cycle

```

1 // Directed version (uncomment commented code for
2 undirected)
3 struct edge {
4     int y;
5     // list<edge>::iterator rev;
6     edge(int _y):y(_y){}
7 };
8 list<edge> g[MAXN];
9 void add_edge(int a, int b){
10     g[a].push_front(edge(b));//auto ia=g[a].begin();
11     // g[b].push_front(edge(a));auto ib=g[b].begin();
12     // ia->rev=ib;ib->rev=ia;

```

```

12 }
13 vector<int> p;
14 void go(int x){
15     while(SZ(g[x])){
16         int y=g[x].front().y;
17         //g[y].erase(g[x].front().rev);
18         g[x].pop_front();
19         go(y);
20     }
21     p.pb(x);
22 }
23 vector<int> get_path(int x){ // get a path that
24     // begins in x
25     // check that a path exists from x before calling
26     // to get_path!
27     p.clear();go(x);reverse(all(p));
28     return p;
29 }

```

5.10 Floyd Warshall

```

1 void floyd(vector<vector<int>>&matriz, ll n) {
2     forn(k,n) forn(i,n) forn(j,n) {
3         matriz[i][j]=min(matriz[i][j],matriz[i][k]+
4             matriz[k][j]);
5     } // O(n^3)
6 }

```

5.11 Fordfulkerson

```

1 struct FordFulkerson {
2     // Time complexity ((V+E) * MAXFLOW)
3     vector < vector<arista> > g;
4     map < pair<int,int>,int > cap;
5     FordFulkerson(int n) {
6         g.resize(n+5);
7     }
8     void add_edge(int x, int y, int z) {
9         g[x].pb({y});
10        g[y].pb({x});
11        cap[{x,y}] = z;
12    } // cap[{x,y}] += z; if multiedges allowed
13    void DFS(int cur, vector<int> &parent, vector<
14        int> &flow, int prev, int cur_flow = INF) {
15        parent[cur] = prev;
16        flow[cur] = cur_flow;
17        for(auto [next] : g[cur]) {
18            if(parent[next] != -1 or cap[{cur,next}] ==
19                0) continue;
20            DFS(next,parent,flow,cur,min(cur_flow,cap[{
21                cur,next}]));
22        }
23    }
24    int max_flow(int s, int t) {
25        int maxflow = 0;
26        vector<int> parent(MAXN,-1), flow(MAXN,-1);
27        DFS(s,parent,flow,s);
28        while(flow[t] > 0) {
29            int new_flow = flow[t];
30            maxflow += new_flow;
31            int cur = t;
32            while(cur != s) {
33                int prev = parent[cur];
34                cap[{prev,cur}] -= new_flow;
35                cap[{cur,prev}] += new_flow;
36                cur = prev;
37            }
38        }
39        return maxflow;
40    }
41 }

```

```

34 }
35 fill(all(parent),-1);
36 fill(all(flow),0);
37 DFS(s,parent,flow,s);
38 }
39 return maxflow;
40 }
41 };

```

5.12 Hungarian

```

1 // O(n^3) assignment problem
2 typedef long double td; typedef vector<td> vd;
3 const td INF=1e100;//for maximum set INF to 0, and
4 //negate costs
5 bool zero(td x){return fabs(x)<1e-9;}//change to x
6 //==0, for ints/ll
7 struct Hungarian{
8     int n; vector<vd> cs; vi L, R;
9     Hungarian(int N, int M):n(max(N,M)),cs(n,vd(n))
10        ,L(n),R(n){
11        forn(x,N)forn(y,M)cs[x][y]=INF;
12    }
13    void set(int x,int y,td c){cs[x][y]=c;}
14    td assign() {
15        int mat = 0; vd ds(n), u(n), v(n); vi dad(n)
16        , sn(n);
17        forn(i,n)u[i]=*min_element(all(cs[i]));
18        forn(j,n){v[j]=cs[0][j]-u[0];forr(i,1,n)v[j]
19            ]=min(v[j],cs[i][j]-u[i]);}
20        L=R=vi(n, -1);
21        forn(i,n)forn(j,n)
22            if(R[j]==-1&&zero(cs[i][j]-u[i]-v[j])){
23                L[i]=j;R[j]=i;mat++;break;}
24        for(;mat<n;mat++){
25            int s=0, j=0, i;
26            while(L[s] != -1)s++;
27            fill(all(dad),-1);fill(all(sn),0);
28            forn(k,n)ds[k]=cs[s][k]-u[s]-v[k];
29            for(;;){
30                j = -1;
31                forn(k,n)if(!sn[k]&&(j==-1||ds[k]<
32                    ds[j]))j=k;
33                sn[j] = 1; i = R[j];
34                if(i == -1) break;
35                forn(k,n)if(!sn[k]){
36                    auto new_ds=ds[j]+cs[i][k]-u[i]
37                        -v[k];
38                    if(ds[k] > new_ds){ds[k]=new_ds
39                        ;dad[k]=j;}
40                }
41            }
42            forn(k,n)if(k!=j&&sn[k]){auto w=ds[k]-
43                ds[j];v[k]+=w,u[R[k]]-=w;}
44            u[s] += ds[j];
45            while(dad[j]>=0){int d = dad[j];R[j]=R[
46                d];L[R[j]]=j;j=d;}
47            R[j]=s;L[s]=j;
48        }
49        td value=0;forn(i,n)value+=cs[i][L[i]];
50        return value;
51    }
52 }
53 };

```

5.13 Max Flow Min Cost Multiedge

```

1 typedef long long tipo;
2 typedef long long tipo_cost;
3 const int MAXN = 505;
4 tipo INF = (tipo)(1e9+7);
5 tipo_cost MAX_COST = (tipo_cost)(1e16);
6 struct arista {
7     int v;
8     tipo cap;
9     tipo_cost c; // Cost
10    int id;
11 };
12 struct prev_node {
13     int prev, id;
14     void setting() {prev = -1, id = -1;}
15 };
16 vector < vector<arista> > g(MAXN);
17 vector <tipo> cap;
18 void add_edge(int x, int y, tipo z, tipo_cost c) {
19     int id = cap.size();
20     g[x].pb({y,z,c,id});
21     g[y].pb({x,0,-1*c,id+1});
22     cap.pb(z); cap.pb(0);
23 }
24 void Bellman_Ford(int s, int t, vector <prev_node>
    &parent, vector <tipo_cost> &d) {
25     for(prev_node u : parent) u.setting();
26     fill(all(d),MAX_COST);
27     vector <bool> inq(MAXN,false);
28     queue <int> q; q.push(s); parent[s] = {s,-1}; inq
        [s] = true; d[s] = 0;
29     while(q.size() > 0) {
30         int u = q.front(); q.pop(); inq[u] = false;
31         for(arista next : g[u]) {
32             if(cap[next.id]>0 && d[next.v] > d[u] + next.
                c) {
33                 d[next.v] = d[u] + next.c;
34                 parent[next.v] = {u,next.id};
35                 if(!inq[next.v]) {
36                     inq[next.v] = true;
37                     q.push(next.v);
38                 }
39             }
40         }
41     }
42 }
43 tipo_cost max_flow_min_cost(int s, int t, int total
    ) {
44     tipo flow = 0;
45     tipo_cost min_cost = 0;
46     vector <prev_node> parent(MAXN);
47     vector <tipo_cost> d(MAXN);
48     while(true) {
49         Bellman_Ford(s,t,parent,d);
50         if(d[t] == MAX_COST) break;
51         tipo new_flow = INF;
52         int cur = t;
53         while(cur != s) {
54             new_flow = min(new_flow, cap[parent[cur].id]);
55             cur = parent[cur].prev;
56         }
57         flow += new_flow;
58         min_cost += d[t] * (tipo_cost)(new_flow);
59         cur = t;
60         while(cur != s) {
61             int prev = parent[cur].prev;

```

```

62         int id = parent[cur].id;
63         cap[id] -= new_flow;
64         cap[(id^1)] += new_flow;
65         cur = prev;
66     }
67 }
68 if(flow < total) return -1;
69 return min_cost;
70 }

```

5.14 Min Cost Flow

```

1 /* O(n^3 m). Usa Dijkstra multiedge, pero no ciclos
    neg de costo*/
2 typedef ll tf; // tipo que se usa para el flow
3 typedef ll tc; // tipo que se usa para el cost
4 const tf INFFLOW=1e9;
5 const tc INFCOST=1e9;
6 struct MCF {
7     int n;
8     vector<tc> prio, pot; vector<tf> curflow; vector<
        int> prevedge, prevnode;
9     priority_queue<pair<tc, int>, vector<pair<tc, int
        >>, greater<pair<tc, int>>> q;
10    struct edge{int to, rev; tf f, cap; tc cost;};
11    vector<vector<edge>> g;
12    MCF(int _n):n(_n),prio(_n),pot(_n),curflow(_n),
        prevedge(_n),prevnode(_n),g(_n){}
13    void add_edge(int s, int t, tf cap, tc cost) {
14        g[s].pb({t,SZ(g[t]),0,cap,cost});
15        g[t].pb({s,SZ(g[s])-1,0,0,-cost});
16    }
17    pair<tf,tc> get_flow(int s, int t) {
18        tf flow=0; tc flowcost=0;
19        while(1){
20            q.push({0, s});
21            fill(all(prio),INFCOST);
22            prio[s]=0; curflow[s]=INFFLOW;
23            while(!q.empty()) {
24                auto cur=q.top(); q.pop();
25                tc d=cur.first; int u=cur.second;
26                if(d!=prio[u]) continue;
27                for(i, SZ(g[u])){
28                    edge &e=g[u][i];
29                    int v=e.to;
30                    if(e.cap<=e.f) continue;
31                    tc nprio=prio[u]+e.cost+pot[u]-pot[v];
32                    if(prio[v]>nprio) {
33                        prio[v]=nprio;
34                        q.push({nprio, v});
35                        prevnode[v]=u; prevedge[v]=i;
36                        curflow[v]=min(curflow[u], e.cap-e.f);
37                    }
38                }
39            }
40            if(prio[t]==INFCOST) break;
41            for(i, n) pot[i]+=prio[i];
42            tf df=min(curflow[t], INFFLOW-flow);
43            flow+=df;
44            for(int v=t; v!=s; v=prevnode[v]) {
45                edge &e=g[prevnode[v]][prevedge[v]];
46                e.f+=df; g[v][e.rev].f-=df;
47                flowcost+=df*e.cost;
48            }
49        }

```

```

50     return {flow,flowcost};
51 }
52 };

```

5.15 Mst Directed

```

1 struct Edge { int a, b; ll w; };
2 struct Node { /// lazy skew heap node
3     Edge key;
4     Node *l, *r;
5     ll delta;
6     void prop() {
7         key.w += delta;
8         if (l) l->delta += delta;
9         if (r) r->delta += delta;
10        delta = 0;
11    }
12    Edge top() { prop(); return key; }
13 };
14 Node *merge(Node *a, Node *b) {
15     if (!a || !b) return a ? b;
16     a->prop(), b->prop();
17     if (a->key.w > b->key.w) swap(a, b);
18     swap(a->l, (a->r = merge(b, a->r)));
19     return a;
20 }
21 void pop(Node*& a) { a->prop(); a = merge(a->l, a->r); }
22 pair<ll, vi> dmst(int n, int r, vector<Edge>& g) {
23     UnionFind uf(n);
24     vector<Node*> heap(n);
25     for (Edge e : g) heap[e.b] = merge(heap[e.b], new Node{e});
26     ll res = 0;
27     vi seen(n, -1), path(n), par(n);
28     seen[r] = r;
29     vector<Edge> Q(n), in(n, {-1,-1}), comp;
30     deque<tuple<int, int, vector<Edge>>> cys;
31     forr(s,0,n) {
32         int u = s, qi = 0, w;
33         while (seen[u] < 0) {
34             if (!heap[u]) return {-1,{}};
35             Edge e = heap[u]->top();
36             heap[u]->delta -= e.w, pop(heap[u]);
37             Q[qi] = e, path[qi++] = u, seen[u] = s;
38             res += e.w, u = uf.find(e.a);
39             if (seen[u] == s) { /// found cycle, contract
40                 Node* cyc = 0;
41                 int end = qi, time = uf.snap();
42                 do cyc = merge(cyc, heap[w = path[--qi]]);
43                 while (uf.join(u, w));
44                 u = uf.find(u), heap[u] = cyc, seen[u] = -1;
45                 cys.push_front({u, time, {&Q[qi], &Q[end]}});
46             }
47         }
48         forr(i,0,qi) in[uf.find(Q[i].b)] = Q[i];
49     }
50     for (auto& [u,t,comp] : cys) { // restore sol (optional)
51         uf.rollback(t);
52         Edge inEdge = in[u];
53         for (auto& e : comp) in[uf.find(e.b)] = e;
54         in[uf.find(inEdge.b)] = inEdge;

```

```

55     }
56     forr(i,0,n) par[i] = in[i].a;
57     return {res, par};
58 }

```

5.16 Toposort

```

1 vi tsort(vector<vi> &g, int n){
2     vi r; priority_queue<ll> q;
3     vi d(2*n);
4     forn(i, n) forn(j: g[i]) d[j]++;
5     forn(i, n) if(!d[i]) q.push(-i);
6     while(!q.empty()){
7         ll x=-q.top(); q.pop(); r.pb(x);
8         forn(j: g[x]){
9             d[j]--;
10            if(!d[j]) q.push(-j);
11        }
12    }
13    return r;
14 } // lexicographically smallest

```

6 Math

Números de Bell (B_n)

Particiones de un conjunto de n elementos. $B_0 = 1$.

Recursiva:

$$B_{n+1} = \sum_{k=0}^n \binom{n}{k} B_k$$

Números de Catalan (C_n)

Paréntesis, triangulaciones, árboles binarios. $C_0 = 1$.

Explícita:

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \frac{(2n)!}{(n+1)!n!}$$

6.1 Fft Mod

```

1 typedef double ld;
2 typedef complex<ld> cd;
3 void fft(vector<cd> &a){
4     int n=SZ(a), L=31-__builtin_clz(n);
5     vi rev(n);
6     forn(i, n) rev[i]=(rev[i/2]+((i&1)<<L))/2;
7     forn(i, n) if(i<rev[i]) swap(a[i],a[rev[i]]);
8     static vector<cd> root(2,1);
9     for(static int k=2;k<n;k*=2){
10        root.resize(n);
11        cd z=polar(1.0,acos(-1)/k);
12        forr(i, k, 2*k) root[i]=root[i/2]*((i&1)?z:1);
13    }
14    for(int k=1;k<n;k*=2){
15        for(int i=0;i<n;i+=2*k){
16            forn(j, k){
17                cd z=root[j+k]*a[i+j+k];
18                a[i+j+k]=a[i+j]-z;
19                a[i+j]+=z;
20            }
21        }
22    }

```

```

23 }
24 vi mul(vi &a, vi &b){
25     int sa=SZ(a), sb=SZ(b);
26     if(sa==0||sb==0) return {};
27     int n=1<<(32-__builtin_clz(sa+sb-2));
28     vector<cd> f(n),h(n);
29     forn(i, n) f[i]=cd(ld(i<sa?a[i]:0),ld(i<sb?b[i]
30         ):0));
31     fft(f);
32     forn(i, n){
33         ll j=(-i)&(n-1);
34         h[i]=(f[j]*f[j]-conj(f[i]*f[i]))*cd
35             (0,-0.25/n);
36     }
37     fft(h);
38     vi c(sa+sb-1);
39     forn(i, sa+sb-1) c[i]=ll(round(h[i].real()));
40     return c;
41 }
42 vi mul_mod(vi &a, vi &b, ll mod){
43     int sa=SZ(a), sb=SZ(b);
44     if(sa==0||sb==0) return {};
45     vi a1(sa),a2(sa),b1(sb),b2(sb);
46     forn(i, sa) a1[i]=(a[i]&((1<<15)-1)),a2[i]=(a[i]
47         ]>>15);
48     forn(i, sb) b1[i]=(b[i]&((1<<15)-1)),b2[i]=(b[i]
49         ]>>15);
50     vi c1=mul(a1,b1),c2=mul(a1,b2),c3=mul(a2,b1),c4
51         =mul(a2,b2);
52     vi c(sa+sb-1);
53     forn(i, sa+sb-1) c[i]=(c1[i]+((1<<15)*(c2[i]%
54         mod+c3[i]%mod))+((1<<30)*(c4[i]%mod)))%mod;
55     return c;
56 }

```

6.2 Fht

```

1 ll c1[MAXN+9],c2[MAXN+9]; // MAXN=pow2
2 void fht(ll* p, int n, bool inv){
3     for(int l=1;2*l<=n;l*=2)for(int i=0;i<n;i+=2*l)
4         forn(j,l){
5             ll u=p[i+j],v=p[i+l+j];
6             if(!inv)p[i+j]=u+v,p[i+l+j]=u-v; // XOR
7             else p[i+j]=(u+v)/2,p[i+l+j]=(u-v)/2;
8             //if(!inv)p[i+j]=v,p[i+l+j]=u+v; // AND
9             //else p[i+j]=-u+v,p[i+l+j]=u;
10            //if(!inv)p[i+j]=u+v,p[i+l+j]=u; // OR
11            //else p[i+j]=v,p[i+l+j]=u-v;
12        }
13 }
14 vi multiply(vi &p1, vi &p2){
15     int n=1<<(32-__builtin_clz(max(SZ(p1),SZ(p2))
16         -1));
17     forn(i,n)c1[i]=0,c2[i]=0;
18     forn(i,SZ(p1))c1[i]=p1[i];
19     forn(i,SZ(p2))c2[i]=p2[i];
20     fht(c1,n,false);fht(c2,n,false);
21     forn(i,n)c1[i]*=c2[i];
22     fht(c1,n,true);
23     return vi(c1,c1+n);
24 }

```

6.3 Gauss Sistema Ecuaciones

```

1 /* Resuelve Ax=B. filas ecuaciones, columnas
   variables.

```

```

2 En ans solucion unica. A es de n x m.
3 n ecuaciones y m-1 variables, ultima columna matriz
4 B.
5 Complejidad O(min(n, m)nm). Si n == m, es O(n^3).
6 */
7 ll mod(ll x) {x%=MOD; if(x<0) x+=MOD; return x;}
8 ll add(ll a, ll b){return mod(a+b);}
9 ll sub(ll a, ll b){return mod(a-b);}
10 ll mul(ll a, ll b){return mod(mod(a)*mod(b));}
11 ll be(ll a, ll p) {
12     ll ans=1; for(; p; p/=2, a = mul(a,a)) if(p&1)
13         ans=mul(ans,a);
14     return ans;
15 }
16 ll divi(ll a, ll b){return mul(a,be(b,MOD-2));}
17 int gauss(vector<vi> &a, vi &ans) {
18     int n = SZ(a), m = SZ(a[0])-1;
19     vector<int> where(m, -1);
20     for(int col=0, row=0; col<m && row<n; ++col) {
21         int sel = row;
22         forr(i, row, n){ // if(abs(a[i][col]) > abs
23             (a[sel][col]))
24             if(a[i][col] > a[sel][col]) sel = int(i
25                 );
26         }
27         if(a[sel][col] == 0) continue; // if(abs(a[
28             sel][col]) < EPS)
29         forr(i, col, m+1) swap(a[sel][i], a[row][i
30             ]);
31         where[col] = row;
32         forn(i, n){
33             if(i != row) {
34                 ll c = divi(a[i][col], a[row][col])
35                     ; // double c = a[i][col] / a[
36                     row][col];
37                 forr(j, col, m+1) a[i][j] = sub(a[i
38                     ][j], mul(a[row][j], c));
39             }
40         }
41         ++row;
42     }
43     ans.assign(m, 0);
44     forn(i, m) if(where[i] != -1) ans[i] = divi(a[
45         where[i]][m], a[where[i]][i]);
46     forn(i, n){
47         ll sum = 0; // double sum = 0;
48         forn(j, m) sum = add(sum, mul(ans[j], a[i][
49             j]));
50         if(sum - a[i][m] != 0) return 0; // if(abs(
51             sum-a[i][m]) > EPS)
52         // No hay soluciones
53     }
54     forn(i, m) if(where[i] == -1) return INF;
55     return 1; // Exactamente una solucion
56 }
57 // Encuentra las bases que generan la solucion
58 vector<vi> find_bases(vector<vi> &A, vi &ans) {
59     int n = SZ(A), m = SZ(ans);
60     vi var_ind;
61     forn(i,SZ(ans)) if(ans[i] == 0) var_ind.pb(i);
62     vector<vi> bases(SZ(var_ind),vi(m,0));
63     int r = 0;
64     for(auto u : var_ind) {
65         bases[r][u] = 1;
66         forn(i,n) {

```

```

54     forn(j,m) {
55         if(A[i][j] != 0) {
56             bases[r][j] = divi(-A[i][u], A[i][j]);
57             break;
58         }
59     }
60 }
61 r++;
62 }
63 return bases;
64 }

```

6.4 Inverse Matrix

```

1 // modular functions in gauss
2 pair<ll,vector<vi>> inverse_matrix(vector<vector<ll
   >> &x) {
3     // returns det and the inverse of the matrix,
4     // if det == 0, no inverse
5     int n = SZ(x); vector <vi> I(n,vi(n,0));
6     ll det = 1;
7     forn(i,n) I[i][i] = 1;
8     forn(i,n) {
9         int pos = -1;
10        forr(fila,i,n) if(x[fila][i] > 0) pos =
            fila;
11        if(pos == -1) {return {0,I};}
12        if(pos != i) {
13            swap(x[i],x[pos]), swap(I[i],I[pos]);
14            det = mul(det,-1);
15        }
16        ll pivot = x[i][i];
17        det = mul(det,pivot);
18        forn(j,n) {
19            x[i][j] = divi(x[i][j],pivot);
20            I[i][j] = divi(I[i][j],pivot);
21        }
22        forn(fila,n) {
23            if(fila == i) continue;
24            pivot = x[fila][i];
25            forn(j,n) {
26                x[fila][j] = sub(x[fila][j],mul(pivot,x[i][
                    j]));
27                I[fila][j] = sub(I[fila][j],mul(pivot,I[i][
                    j]));
28            }
29        }
30    }
31    return {det,I};
32 }

```

6.5 Lagrange

```

1 ll lagrange(vii a, ll x){
2     ll ans = 0;
3     int n = SZ(a);
4     forn(i, n){
5         ll term = a[i].second;
6         forn(j, n){
7             if(j != i)
8                 term = term*(x-a[j].first)/(a[i].first-a[j
                ].first);
9         }
10        ans += term;
11    }
12    return ans;

```

```

13 }

```

6.6 Linear Rec

```

1 const ll LOG = 60;
2 // Linear Recurrence
3 // Description: Calculates the kth term of a linear
   recurrence relation
4 // init O(n^2log) query(n^2 logk)
5 // input: terms: first n term; trans: transition
   function (calcular con BM); MOD; LOG=mxlog of k
6 // output calc(k): kth term mod MOD
7 // example: {1,1} {2,1} an=2*a_(n-1)+a_(n-2); calc
   (3)=3 calc(10007)=71480733
8 struct LinearRec {
9     ll n; vi terms, trans; vector<vi> bin;
10    vi add(vi &a, vi &b){
11        vi res(n*2+1);
12        forn(i, n+1) forn(j, n+1) res[i+j]=(res[i+j]
            ]*1LL+(1ll)a[i]*b[j])%MOD;
13        for(ll i=2*n; i>n; --i){
14            forn(j, n) res[i-1-j]=(res[i-1-j]*1LL+(
                1ll)res[i]*trans[j])%MOD;
15            res[i]=0;
16        }
17        res.erase(res.begin()+n+1,res.end());
18        return res;
19    }
20    LinearRec(vi &terms, vi &trans):terms(terms),
        trans(trans){
21        n=SZ(trans);vi a(n+1);a[1]=1;
22        bin.pb(a);
23        forr(i,1,LOG)bin.pb(add(bin[i-1],bin[i-1]))
            ; // Precompute powers of a
24    }
25    ll calc(ll k){
26        vi a(n+1);a[0]=1;
27        forn(i,LOG) if((k>>i)&1)a=add(a,bin[i]);
28        ll ret=0;
29        forn(i,n) ret=(ret+a[i+1]*terms[i])%MOD;
30        return ret;
31    }
32 };

```

6.7 Matrix Fast Pow

```

1 typedef vector<vector<ll>> Matrix;
2 Matrix ones(int n) {
3     Matrix r(n,vector<ll>(n));
4     forn(i,n)r[i][i]=1;
5     return r;
6 }
7 Matrix operator*(Matrix &a, Matrix &b) {
8     int n=a.size(),m=b[0].size(),z=a[0].size();
9     Matrix r(n,vector<ll>(m));
10    forn(i,n)forn(j,m)forn(k,z)
11        r[i][j]+=a[i][k]*b[k][j],r[i][j]%MOD;
12    return r;
13 }
14 Matrix be(Matrix b, ll e) {
15     Matrix r=ones(b.size());
16     while(e){if(e&1LL)r=r*b;b=b*b;e/=2;}
17     return r;
18 }

```

6.8 Matrix Reduce

```

1 double reduce(vector<vector<double> > &x) {
2     int n=x.size(),m=x[0].size();
3     int i=0,j=0;double r=1.;
4     while(i<n&&j<m){
5         int l=i;
6         forr(k,i+1,n)if(abs(x[k][j])>abs(x[l][j]))l
            =k;
7         if(abs(x[l][j])<EPS){j++;r=0.;continue;}
8         if(l!=i){r=-r;swap(x[i],x[l]);}
9         r*=x[i][j];
10        for(int k=m-1;k>=j;k--)x[i][k]/=x[i][j];
11        forr(k,0,n){
12            if(k==i)continue;
13            for(int l=m-1;l>=j;l--)x[k][l]-=x[k][j]
                *x[i][l];
14        }
15        i++;j++;
16    } // returns determinant
17    return r;
18 }

```

6.9 Print Int128

```

1 string toString(__int128 num) {
2     string str;
3     do {
4         int digit = num % 10;
5         str = to_string(digit) + str;
6         num = (num - digit) / 10;
7     } while (num != 0);
8     return str;
9 }
10 ostream& operator<<(std::ostream& os, __int128 t) {
11     string str = toString(t);
12     return os << str;
13 }

```

6.10 Random

```

1 mt19937_64 rng(chrono::steady_clock::now().
    time_since_epoch().count());
2 template<class T>
3 T rnd(T l, T r) { return uniform_int_distribution<T>
    (>(l, r)(rng); }
4 using u128 = __uint128_t;
5 u128 rnd128() { return (u128(rng()) << 64) | rng();
    }
6 shuffle(all(v), rng);

```

6.11 Simplex

```

1 struct Simplex {
2     vector<int> X,Y;
3     vector<vector<double>> A;
4     vector<double> b,c;
5     double z;
6     int n,m;
7     void add_equation(vector<double> v, double val)
8         { A.pb(v); b.pb(val); }
9     void set_objective(vector<double> _c){ c = _c; }
10    void pivot(int x,int y){
11        swap(X[y],Y[x]);
12        b[x]/=A[x][y];
13        forr(i,0,m)if(i!=y)A[x][i]/=A[x][y];
14        A[x][y]=1/A[x][y];
15        forr(i,0,n)if(i!=x&&abs(A[i][y])>EPS){

```

```

15        b[i]-=A[i][y]*b[x];
16        forr(j,0,m)if(j!=y)A[i][j]-=A[i][y]*A[x]
            [j];
17        A[i][y]=-A[i][y]*A[x][y];
18    }
19    z+=c[y]*b[x];
20    forr(i,0,m)if(i!=y)c[i]-=c[y]*A[x][i];
21    c[y]=-c[y]*A[x][y];
22 }
23 pair<double,vector<double>> simplex() {
24     // maximiza c^T x, dado Ax<=b, x>=0
25     // retorna par (maximum value, solution
        vector)
26     n=b.size();m=c.size();z=0.;
27     X=vector<int>(m);Y=vector<int>(n);
28     forn(i,m)X[i]=i;
29     forn(i,n)Y[i]=i+m;
30     while(1){
31         int x=-1,y=-1;
32         double mn=-EPS;
33         forn(i,n)if(b[i]<mn)mn=b[i],x=i;
34         if(x<0) break;
35         forr(i,m)if(A[x][i]<-EPS){y=i;break;}
36         assert(y>0 && "No solution found");
37         pivot(x,y);
38     }
39     while(1){
40         double mx=EPS;
41         int x=-1,y=-1;
42         forn(i,m)if(c[i]>mx)mx=c[i],y=i;
43         if(y<0)break;
44         double mn=1e200;
45         forn(i,n)if(A[i][y]>EPS&&b[i]/A[i][y]<
            mn)mn=b[i]/A[i][y],x=i;
46         assert(x>0 && "c^T x is unbounded");
47         pivot(x,y);
48     }
49     vector<double> r(m);
50     forn(i,n)if(Y[i]<m)r[Y[i]]=b[i];
51     return mp(z,r);
52 }
53 };

```

7 Number Theory

7.1 Binexp Invmod

```

1 ll be(ll b, ll e, ll m = MOD) {
2     ll r=1; b%=m;
3     while(e){if(e&1LL)r=r*b%m;b=b*b%m;e/=2;}
4     return r;
5 }
6 ll inv_mod(ll x, ll m = MOD) {return be(x,m-2,m);}

```

7.2 Criba Primos

```

1 vi min_prime;
2 vi criba(ll n) {
3     vb prime(n+1,true);
4     min_prime.resize(n+1,INF);
5     vi primos;
6     for(ll p=2; p*p<=n; p++){
7         if(!prime[p]) continue;
8         for(ll i=p*p; i<=n; i += p) {
9             prime[i] = false;

```

```

10     min_prime[i] = min(min_prime[i],p);
11 }
12 }
13 forr(i, 2, n+1) if(prime[i]) primos.pb(i),
14     min_prime[i] = i;
15 return primos;
16 }

```

7.3 Discrete Log

```

1  /* halla x tal que  $a^x = b \pmod m$  en  $O(\sqrt{m})$  */
2  ll discrete_log(ll a, ll b, ll m) {
3      a %= m, b %= m;
4      ll n = sqrt(m) + 1;
5      ll an = 1;
6      for (ll i = 0; i < n; ++i)
7          an = (an * 1ll * a) % m;
8      unordered_map<ll, ll> vals;
9      for (ll q = 0, cur = b; q <= n; ++q) {
10         vals[cur] = q;
11         cur = (cur * 1ll * a) % m;
12     }
13     for (ll p = 1, cur = 1; p <= n; ++p) {
14         cur = (cur * 1ll * an) % m;
15         if (vals.count(cur)) {
16             ll ans = n * p - vals[cur];
17             return ans;
18         }
19     }
20     return -1;
21 }

```

7.4 Discrete Root

```

1  // Finds the primitive root modulo p
2  ll generator(ll p) {
3      vi fact;
4      ll phi = p-1, n = phi;
5      for (ll i = 2; i * i <= n; ++i) {
6          if (n % i == 0) {
7              fact.push_back(i);
8              while (n % i == 0)
9                  n /= i;
10         }
11     }
12     if (n > 1)
13         fact.push_back(n);
14     forr(res, 2, p+1){
15         bool ok = true;
16         for (ll factor : fact) {
17             if (be(res, phi / factor, p) == 1) {
18                 ok = false;
19                 break;
20             }
21         }
22         if (ok) return res;
23     }
24     return -1;
25 }
26 // finds all numbers x such that  $x^k = a \pmod n$ 
27 vi discrete_root(ll n, ll k, ll a){
28     if (a == 0) return {0};
29     ll g = generator(n);
30     ll sq = (ll) sqrt (n + .0) + 1;
31     vector<pair<ll, ll>> dec(sq);
32     forr(i, 1, sq+1)

```

```

33         dec[i-1] = {be(g, i * sq * k % (n - 1), n),
34             i};
35     sort(all(dec));
36     ll any_ans = -1;
37     forn(i, sq){
38         ll my = be(g, i * k % (n - 1), n) * a % n;
39         auto it = lower_bound(all(dec), mp(my, 0));
40         if (it != dec.end() && it->first == my) {
41             any_ans = it->second * sq - i;
42             break;
43         }
44     }
45     if (any_ans == -1) return {};
46     ll delta = (n-1) / __gcd(k, n-1);
47     vi ans;
48     for (ll cur = any_ans % delta; cur < n-1; cur
49         += delta)
50         ans.push_back(be(g, cur, n));
51     sort(ans.begin(), ans.end());
52     return ans;
53 }

```

7.5 Divisors

```

1  vi find_divisors(ll n, vi &primos) {
2      vector<pair<ll,ll>> factor;
3      for(ll prime : primos) {
4          int cont = 0;
5          while(n % prime == 0) {
6              cont++;
7              n /= prime;
8          }
9          if(cont > 0) factor.pb({prime,cont});
10     }
11     if(n > 1) factor.pb({n,1});
12     vi divisores = {1};
13     for(auto [p,exp] : factor) {
14         int tam = SZ(divisores);
15         forn(i,exp) {
16             forn(j,tam) {
17                 int pos = SZ(divisores)-tam;
18                 divisores.pb(divisores[pos] * p);
19             }
20         }
21     }
22     sort(all(divisores));
23     return divisores;
24 }

```

7.6 Fast Factorization Mrabin Prho

```

1  typedef long double ld;
2  mt19937_64 rng(chrono::steady_clock::now().
3      time_since_epoch().count());
4  ll myrand(ll a, ll b) { return
5      uniform_int_distribution<ll>(a, b)(rng); }
6  struct Factorization {
7      inline ll mul(ll a, ll b, ll c){
8          ll s = a * b - c * ll((ld) a / c * b + 0.5)
9          ;
10         return s < 0 ? s + c : s;
11     }
12     ll be(ll a, ll k, ll mod) {
13         ll res = 1;
14         for(; k; k >>= 1, a = mul(a, a, mod)) if (k
15             & 1) res = mul(res, a, mod);
16     }
17 }

```

```

12     return res;
13 }
14 bool miller(ll n) {
15     auto test = [&](ll n, ll a) {
16         if (n == a) return true;
17         if (n % 2 == 0) return false;
18         ll d = (n - 1) >> __builtin_ctzll(n -
19             1);
20         ll r = be(a, d, n);
21         while (d < n - 1 && r != 1 && r != n -
22             1) {
23             d <= 1;
24             r = mul(r, r, n);
25         }
26         return r == n - 1 || d & 1;
27     };
28     if(n == 2) return 1;
29     for(auto p: vi{2, 3, 5, 7, 11, 13}) if(!
30         test(n, p)) return 0;
31     return 1;
32 }
33 ll pollard(ll n) {
34     auto f = [&](ll x) { return ll((__int128(x)
35         * x + 1) % n); };
36     ll x = 0, y = 0, t = 30, prd = 2;
37     while (t++ % 40 || __gcd(prd, n) == 1) {
38         if (x == y) x = myrand(2, n - 1), y = f
39             (x);
40         ll tmp = mul(prd, abs(x - y), n);
41         if (tmp) prd = tmp;
42         x = f(x), y = f(f(y));
43     }
44     return __gcd(prd, n);
45 }
46 vi factorize(ll n) {
47     vi res;
48     auto dfs = [&](auto &dfs, ll x) {
49         if (x == 1) return;
50         if (miller(x)) res.pb(x);
51         else {
52             ll d = pollard(x);
53             dfs(dfs, d); dfs(dfs, x / d);
54         }
55     };
56     dfs(dfs, n); sort(all(res));
57     return res;
58 }
59 };

```

7.7 Teo Chino Resto

```

1 ii extendedEuclid(ll a, ll b){
2     ll x,y; //a * x + b * y = gcd(a,b)
3     if (b==0) return {1, 0};
4     auto p = extendedEuclid(b, a/b);
5     x = p.second, y = p.first-(a/b)*x;
6     if(a*x+b*y == -__gcd(a,b)) x=-x, y=-y;
7     return {x, y};
8 }
9 pair<ii, ii> dioph(ll a, ll b, ll r) {
10     ll d=__gcd(a,b); // a*x+b*y=r
11     a/=d; b/=d; r/=d;
12     auto p = extendedEuclid(a,b);
13     p.first*=r; p.second*=r;
14     assert(a*p.first+b*p.second==r);

```

```

15     return {p, {-b,a}}; // sol=p+t*ans.ss
16 }
17 ll inv(ll a, ll mod) { // inv a mod m
18     assert(__gcd(a,mod)==1);
19     ii sol = extendedEuclid(a,mod);
20     return ((sol.first%mod)+mod)%mod;
21 }
22 ii sol(tuple<ll,ll,ll> c){ //requires inv,
23     diophantine
24     auto [a, x1, m] = c; ll d = __gcd(a,m);
25     if(d==1) return {mod(x1*inv(a,m),m), m};
26     return x1%d ? mp(-1LL,-1LL) : sol({a/d, x1/d, m
27         /d});
28 }
29 // cond[i] = {ai,bi,mi} ai*xi=bi (mi); assumes lcm
30 fits in ll
31 // Mucho cuidado con el overflow, usar __int128 si
32 lcm no entra en ll
33 pair<ll,ll> crt(vector<tuple<ll,ll,ll>> cond) {
34     ll x1=0,m1=1,x2,m2;
35     for(auto t:cond){
36         tie(x2,m2) = sol(t);
37         if((__int128)(x1-x2)%m1==0) return {-1,-1};
38         if(m1==m2) continue;
39         ll k=dioph(m2,-m1,x1-x2).first.second, l=m1
40             *(m2/__gcd(m1,m2));
41         x1 = mod((__int128)m1*k+x1,l); m1=l;
42     } // returns: (sol, lcm)
43     return sol({1,x1,m1});
44 }

```

8 Python

8.1 Numbs

```

1 import random
2 x = random.randint(1,10) # (including 1 and 10)
3 choices = ["apple", "banana", "cherry"]
4 x = random.choice(choices) # Returns random element
5 from list
6 x = 5 % -2 # -1 (because 5 - (-3 * -2) = 5 - 6 =
7     -1)
8 x = -5 % 2 # 1 (because -5 - (-3 * 2) = -5 - (-6)
9     = 1)
10 l = list() # Defining empty list (l = [])
11 l.extend([2,3]) # Extending an existing list
12 l.insert(1, 100) # Inserting at specific index
13 x = l.pop()
14 x = l.pop(2) # Removing from specific index
15 x = l + [4,5] # Merging two lists (does not change
16     existing list)
17 l.remove(2) # Removing first occurrence of an
18     element
19 l = [0,1,2,3,4,5,6]
20 l1 = l[2:4] # Getting sublist, returns [2,3]
21 l1 = l[:] # Quickly make a copy of list
22 l[0], l[2] = l[2], l[0] # Swapping two elements
23 l = list({"a"}) # Initializing, from set -> ["a"]
24 d = {"a": 1, "c": 2}
25 l = list(d) # From dict keys -> ["a", "c"]
26 l = list(d.keys()) # From dict keys -> ["a", "c"]
27 l = list(d.values()) # From dict values -> [1,2]
28 l = list(d.items()) # From dict key-values -> [("a",
29     1), ("c", 2)]
30 l = [1]*n # List with 1 repeated n times

```

```

25 l = list(range(10)) # Returns [0,1,2,3...,8,9]
26 l = [x+1 for x in nums] # Preparing list from
    another iterable
27 nums = [5,1,4,9,8]
28 for i, n in enumerate(nums): # Iterating with index
    print(f"Index: {i} , number: {n}")
29 for i in range(0,n,2): # Iterating over range with
    step size = 2
30     print(i)
31 for i in range(len(nums)): # Option 1
    print(nums[~i])
32 for i in range(len(nums), -1, -1): # Option 2
    print(nums[i])
33 for i in range(len(nums)//2): # Two pointer (left
    and right end) iteration
34     print(i) # 0,1,2
35     print(~i) # -1 (i.e. 4), -2 (i.e. 3), -3(i.e.
        2)
36 for x, y in zip(l1, l2): # Iterate over two lists
    print(x)
    print(y)
37 for i, (x, y) in enumerate(zip(l1,l2)):
    print(f"Index: {i}, x: {x}, y: {y}")
38 nums = [1,2,3,4]
39 for n in nums[:]: # Safely removing element from
    list while iterating
40     if n % 2 == 0:
41         nums.remove(n)
42     nums = [5,1,4,9,8]
43     nums.sort() # In-place sorting
44     nums.sort(reverse=True) # Descending order
45     new_nums = sorted(nums) # Getting new sorted list
46     new_nums = sorted(nums, reverse=True) # Descending
        order
47     nums = [1,-1,3,2,-3,-4]
48     nums.sort(key=abs) # Sort based on absolute value
49     nums = ["apple", "banana", "cherry"]
50     nums.sort(key=len) # Sort based on length
51     nums = [(0,1), (3,1), (1,2)]
52     nums.sort(key = lambda x : x[1]) # Sorts based on
        first element
53     nums = [{"age": 18, "name": "x"}, {"age": 12, "name":
        "y"}]
54     nums.sort(key = lambda x: x["age"]) # Sorts based
        age
55 from collections import Counter
56 c = Counter([1,2,4,1,2,5]) # Counter({1: 2, 2: 2,
    4: 1, 5: 1})
57 for k, v in c.items():
58     print(f"Element: {k}, Frequency: {v}")
59 x = 1 in c # Membership checking
60 c[1] = 1 # Updating frequency
61 del c[1] # Removing character
62 from bisect import bisect_left, bisect_right #
    Equivalent to lower_bound and upper_bound
63 l = [1,2,2,3,4]
64 x = bisect_left(l, -1) # Returns 0
65 x = bisect_right(l, -1) # Returns 0
66 x = bisect_left(l, 10) # Returns 5
67 x = bisect_right(l, 10) # Returns 5
68 x = bisect_left(l, 2) # Returns 1 (index of first
    occurence of 2)
69 x = bisect_right(l, 2) # Returns 3 (1 + index of
    last occurence of 2)
70 output = all(x%2 == 0 for x in nums) # Check if

```

```

    list has all even numbers
71 output = any(x == 0 for x in nums) # Check if list
    has one or more zero value
72 is_pal = all(s[i] == s[-i] for i in range(len(s)
    //2)) # Check if string is palindrome
73 from itertools import permutations, combinations
74 items = ["A", "B", "C"]
75 for c in combinations(items, 2):
76     print(c) # Prints ("A", "B"), ("A", "C"), ("B",
        "C")
77 for p in permutations(items): # Iterate over all
    permutations
78     print(p)
79 for p in permutations(items, 2): # Iterate over all
    permutations of length 2
80     print(p)
81 d = dict() # Defining empty dict, also d = {}
82 d["a"] = 1 # Add/Update value against key
83 x = d["a"] # Raises KeyError if key is not present
84 x = d.get("a", None) # Returns default value if key
    is not present
85 if "a" in d: # Get a value and delete key
    x = d.pop("a")
86 if "a" in d: # Delete key
    del d["a"]
87 sorted_keys = sorted({"x": 1, "a": 2}) # Returns ["
    a", "x"]
88 d.pop(next(iter(d))) # Pop first inserted key
89 from collections import defaultdict
90 d = defaultdict(int) # initializes non-present key
    with value 0
91 d = defaultdict(lambda : 1) # initializes non-
    present key with value 1
92 d = defaultdict(list) # initializes non-present key
    with empty list ([])
93 d = defaultdict(dict) # initializes non-present key
    with empty dict ( {})
94 d = { x : x%2 == 0 for x in nums } # Dictionary
    comprehension : Preparing dict from another
    iterable
95 d = dict(['()', '[]', '{}']) # Returns {'(': ')',
    '[': ']', '{': '}' } # Create a dictionary from
    list/tuple of 2 length strings
96 t = (1, 2, 3) # Initialize tuple
97 t = (1,) # Make sure to add a comma to initialize
    single element tuple correctly # Otherwise it
    is treated as integer
98 t[0] = 4 # As tuples are immutable, this will raise
    a TypeError
99 d = {(1, 2): 'value'} # Tuple as dict keys
100 s = {(1, 2), (3, 4)} # Tuple as set elements
101 def multi_value_func(): # Unpacking
    return 1, 2, 3
102 x, y, z = multi_value_func()
103 s = set() # Defining an empty set
104 s.add(1) # Adding element
105 size = len(s) # Getting size of set
106 s.update([1,2,3]) # Adding multiple elements to set
107 if 1 in s: # Removing already added element
    s.remove(1)
108 else:
109     print("Removing an element that does not exist in
        set, throws KeyError.")
110 s.discard(1) # Removing element irrespective of
    whether it exists in set or not

```

```

121 if len(s) == 0: # Checking whether set is empty
122     print("Set is empty")
123 if not s:
124     print("Set is either empty or s is None")
125 s = set([1,2])
126 for i, x in enumerate(s): # Iterating over set
127     print(f"Insertion Index {i} : Element : {x}")
128 s1 = {1,2,3}
129 s2 = {3,4,5}
130 union_set = s1 | s2 # {1,2,3,4,5}
131 intersection_set = s1 & s2 # {3}
132 diff_set = s1 - s2 # {1,2}
133 sym_diff_set = s1 ^ s2 # {1,2,4,5} (Just like XOR,
    returns what is not present in both)
134 s1 <= s2 # Subset test (True)
135 s1 >= s2 # Superset test (False)
136 stack = [] # In python, List can be used as Stack.
137 stack.append(1) # Push
138 x = stack[-1] # Top / Peek
139 x = stack.pop() # Pop
140 size = len(stack) # Getting size of stack
141 from collections import deque
142 dq = deque() # Defining an empty deque
143 dq.append(1) # Appending element at rear/back
144 dq.appendleft(1) # Appending element at front/head
145 x = dq.pop() # Removing and getting element from
    rear/back
146 x = dq.popleft() # Removing and getting element
    from front/head
147 size = len(dq) # Getting length/size of deque
148 x = dq[-1] # Accessing element at rear
149 x = dq[0] # Accessing element at front
150 x = dq[3] # Accessing element at any index
151 if len(dq) == 0: # Checking whether deque is empty
152     print("Deque is empty")
153 if 1 in dq: # Looking for an element
154     print("1 is part of deque")
155 for i, x in enumerate(dq): # Iterating over deque
156     print(f"Index {i} : Element : {x}")
157 l1 = sorted(dq) # Sorted operation returns new list
    which is sorted

```

8.2 Strings

```

1 sls = s.lstrip() # Removes space from left end
2 srs = s.rstrip() # Removes space from right end
3 ss = s.strip() # Removes space from both ends but
    not from the middle
4 x = "".join(["I", "am", "learning", "python"]) #
    Returns "Iamlearningpython"
5 x = " ".join(["I", "am", "learning", "python"]) #
    With delimiter # Returns "I am learning python"
6 lower_case_s = s.lower()
7 upper_case_s = s.upper()
8 x = s.isalnum() # To check if string has all
    characters as either digit or alphabet
9 x = s.isalpha() # To check if string has all
    alphabets
10 x = s.isnum() # To check if string has all digits
11 x = isspace() # To check if string has all spaces
12 x = ord("b") # Returns 98 # Get unicode/ascii
    representation of character
13 x = s[2:4] # Return "sa" # Slicing
14 x = sorted(s) # Returns ['a', 's', 't', 'u', 'v']
15 x = sorted(s, reverse=True) # Returns ['v', 'u', 't'

```

```

    ', 's', 'a']
16 "a" in "utsav" # Returns True
17 s = "prefix"
18 x = s.startswith("pre") #Returns True
19 x = s.endswith("fix") #Returns True
20 s1 = "hello world programming"
21 subs1 = "world"
22 start_index = s1.find(subs1) # Returns 6

```

9 Strings

9.1 Aho Corasick

```

1 struct vertex {
2     map<char,int> next, go;
3     int p,link,nextleaf;
4     char pch;
5     vector<int> leaf;
6     vertex(int p=-1, char pch=-1,int nextleaf=-1):
7         p(p),pch(pch),link(-1),nextleaf(nextleaf){}
8 };
9 vector<vertex> t;
10 vector <vector <int> > g(MAXN); // Suffix-links
    tree
11 void aho_init(){ //do not forget!!
12     t.clear(); t.pb(vertex());
13 }
14 void add_string(string s, int id) {
15     int v=0;
16     for(char c:s) {
17         if(!t[v].next.count(c)){
18             t[v].next[c]=t.size();
19             t.pb(vertex(v,c));
20         }
21         v=t[v].next[c];
22     }
23     t[v].leaf.pb(id);
24 }
25 int go(int v, char c);
26 int get_link(int v) {
27     if(t[v].link < 0) {
28         if(v == 0 || t[v].p == 0) t[v].link = 0;
29         else t[v].link = go(get_link(t[v].p),t[v].
            pch);
30         g[t[v].link].pb(v);
31     }
32     return t[v].link;
33 }
34 int go(int v, char c){
35     if(!t[v].go.count(c)) {
36         if(t[v].next.count(c))t[v].go[c]=t[v].next[
            c];
37         else t[v].go[c] = v == 0 ? 0 : go(get_link(
            v),c);
38     }
39     return t[v].go[c];
40 }
41 int init_next_leaf(int v) {
42     if(v == 0) t[v].nextleaf=0;
43     if(t[get_link(v)].leaf.size()) return t[v].
        nextleaf=get_link(v);
44     else return t[v].nextleaf = t[get_link(v)].
        nextleaf != -1 ?
45         t[get_link(v)].nextleaf :
46         init_next_leaf(get_link(v));

```

```

47 }
48 void construct_links() { forn(i,t.size()) get_link(
    i); }

```

9.2 Dynamic Hash

```

1 struct BIT {
2     vector<ll> prefix, a; ll M;
3     BIT() {}
4     BIT(vector<ll> &v, ll MOD) {
5         int n = v.size(); prefix.resize(n+1); a = v; M
            = MOD;
6         vector<ll> aux(n+1,0);
7         forn(i,n) aux[i+1] = (aux[i] + v[i]) % M;
8         forr(i,1,n+1) prefix[i] = (aux[i] + M - aux[i -
            i&(-i)]) % M;
9     }
10    ll query(int l, int r) { //[a,b] 0-indexed
11        ll ans = 0; r++;
12        while(r) ans += prefix[r], r -= r&(-r), ans %=
            M;
13        while(l) ans += M - prefix[l], l -= l&(-l), ans
            %= M;
14        return ans;
15    }
16    void update(int pos, ll val) {
17        int i = pos + 1; ll upd = (val + M - a[pos]) %
            M;
18        while(i < prefix.size()) prefix[i] += upd,
            prefix[i] %= M, i += i&(-i);
19        a[pos] = val;
20    }
21 };
22 struct Hashing {
23     int n;
24     const ll MOD[2] = {999727999, 1070777777};
25     vector<ll> prefix[2], rev[2], pot[2], inv_pot
        [2];
26     ll P = 31, invP[2] = {inv_mod(P,MOD[0]), inv_mod(
        P,MOD[1])};
27     BIT s[2], rs[2];
28     Hashing() {}
29     Hashing(string &pal) {
30         n = pal.size();
31         forn(k,2) {
32             prefix[k].resize(n,0); rev[k].resize(n,0);
33             pot[k].resize(n,1); inv_pot[k].resize(n,1);
34             forn(i,n) {
35                 if(i) pot[k][i] = (pot[k][i-1] * P) % MOD[k]
                    ];
36                 if(i) inv_pot[k][i] = (inv_pot[k][i-1] *
                    invP[k]) % MOD[k];
37                 prefix[k][i] = (ll)(pal[i]-'a') * pot[k][i]
                    % MOD[k];
38                 rev[k][i] = (ll)(pal[n-i-1]-'a') * pot[k][i]
                    % MOD[k];
39             }
40             s[k] = BIT(prefix[k],MOD[k]);
41             rs[k] = BIT(rev[k],MOD[k]);
42         }
43     }
44     pair<ll,ll> get(int l, int r) { //[l,r] 0-indexed
45         ll x = (s[0].query(l,r) * inv_pot[0][l]) % MOD
            [0];
46         ll y = (s[1].query(l,r) * inv_pot[1][l]) % MOD

```

```

        [1];
47         return {x,y};
48     }
49     pair<ll,ll> getr(int l, int r) { //[l,r] 0-
        indexed
50         l = n-1-l, r = n-1-r; swap(l,r);
51         ll x = (rs[0].query(l,r) * inv_pot[0][l]) % MOD
            [0];
52         ll y = (rs[1].query(l,r) * inv_pot[1][l]) % MOD
            [1];
53         return {x,y};
54     }
55     void update(int pos, char a) {
56         ll val = (ll)(a-'a');
57         forn(k,2) {
58             s[k].update(pos,(val * pot[k][pos]) % MOD[k])
                ;
59             assert(n-1-pos >= 0);
60             rs[k].update(n-1-pos,(val * pot[k][n-1-pos])
                % MOD[k]);
61         }
62     }
63 };

```

9.3 Hashing

```

1 struct Hash {
2     const ll P=17777771;
3     const ll MOD[2] = {999727999, 1070777777};
4     const ll PI[2] = {325255434, 10018302};
5     vector<ll> h[2],pi[2];
6     Hash(string& s){
7         forn(k,2)h[k].resize(s.size()+1),pi[k].
            resize(s.size()+1);
8         forn(k,2){
9             h[k][0]=0;pi[k][0]=1;
10            ll p=1;
11            forr(i,1,s.size()+1){
12                h[k][i]=(h[k][i-1]+p*s[i-1])%MOD[k]
                    ];
13                pi[k][i]=(1LL*pi[k][i-1]*PI[k])%MOD
                    [k];
14                p=(p*P)%MOD[k];
15            }
16        }
17    }
18    ll get(ll s, ll e){ // [s, e] (s y e van de 0 a
        n-1)
19        e++;
20        ll h0=(h[0][e]-h[0][s]+MOD[0]);
21        h0=(1LL*h0*pi[0][s])%MOD[0];
22        ll h1=(h[1][e]-h[1][s]+MOD[1]);
23        h1=(1LL*h1*pi[1][s])%MOD[1];
24        return (h0<<32)|h1;
25    }
26 };

```

9.4 Manacher

```

1 vi d1; // impar
2 vi d2; // par
3 //s aabbaacaabbaa
4 //d1 111111711111
5 //d2 0103010010301
6 void manacher(string& s){
7     ll l=0,r=-1,n=SZ(s);

```

```

8   d1.resize(n), d2.resize(n);
9   forn(i, n){
10      ll k = (i>r ? 1 : min(d1[l+r-i],r-i));
11      while(i+k<n && i-k>=0 && s[i+k]==s[i-k]) k
12          ++;
13      d1[i] = k--;
14      if(i+k>r) l=i-k,r=i+k;
15  }
16  l=0; r=-1;
17  forn(i, n){
18      ll k = (i>r ? 0 : min(d2[l+r-i+1],r-i+1));
19      k++;
20      while(i+k<=n && i-k>=0 && s[i+k-1]==s[i-k])
21          k++;
22      d2[i] = --k;
23      if(i+k-1>r) l=i-k, r=i+k-1;
24  }

```

9.5 Prefix Function

```

1  vector<int> prefix_function(string s) {
2      int n = (int)s.length();
3      vector<int> pi(n);
4      forr(i,1,n) {
5          int j = pi[i-1];
6          while(j > 0 && s[i] != s[j]) j = pi[j-1];
7          if(s[i] == s[j]) j++;
8          pi[i] = j;
9      }
10     return pi;
11 }

```

9.6 Suffix Array

```

1  void csort(vector<int>& sa, vector<int>& r, int k){
2      int n=sa.size();
3      vector<int> f(max(255,n),0),t(n);
4      forr(i,0,n)f[RB(i+k)]++;
5      int sum=0;
6      forr(i,0,max(255,n))f[i]=(sum+=f[i])-f[i];
7      forr(i,0,n)t[f[RB(sa[i]+k)]]+=sa[i];
8      sa=t;
9  }
10 vector<int> suffix_array(string& s){ // O(n logn)
11     s += '$';
12     int n=s.size(),rank;
13     vector<int> sa(n),r(n),t(n);
14     forr(i,0,n)sa[i]=i,r[i]=s[i];
15     for(int k=1;k<n;k*=2){
16         csort(sa,r,k);csort(sa,r,0);
17         t[sa[0]]=rank=0;
18         forr(i,1,n){
19             if(r[sa[i]]!=r[sa[i-1]]||RB(sa[i]+k)!=
20                 RB(sa[i-1]+k))rank++;
21             t[sa[i]]=rank;
22         }
23         r=t;
24         if(r[sa[n-1]]==n-1)break;
25     }
26     return sa;
27 }
28 vector<int> lcp_construction(string const& s,
29     vector<int> const& p) {
30     int n = s.size();
31     vector<int> rank(n, 0);

```

```

32     for (int i = 0; i < n; i++) rank[p[i]] = i;
33     int k = 0;
34     vector<int> lcp(n-1, 0);
35     for (int i = 0; i < n; i++) {
36         if(rank[i] == n - 1) {
37             k = 0;
38             continue;
39         }
40         int j = p[rank[i] + 1];
41         while (i + k < n && j + k < n && s[i+k] ==
42             s[j+k]) k++;
43         lcp[rank[i]] = k;
44         if(k) k--;
45     }
46     return lcp;
47 }
48 bool substr_search(string &text, string &pal,
49     vector<int> &sa) {
50     int n = text.size(), a = -1, b = n-1;
51     while(b-a > 1) {
52         int med = (a+b)/2, pos = sa[med]; bool d =
53             false;
54         int tam = min((int)pal.size(),int(n-sa[med]
55             ));
56         string check = text.substr(sa[med],tam);
57         if(check < pal) a=med;
58         else b = med;
59     }
60     int tam = min(int(pal.size()),int(n-sa[b]));
61     string fin = text.substr(sa[b],tam);
62     return fin == pal ? true : false;
63 }
64 ll count_substring(string &pal) {
65     ll n = pal.size();
66     vector<int> sa = suffix_array(pal);
67     vector<int> lcp = lcp_construction(pal,sa);
68     ll ans = n * (n+1) / 2;
69     for(int u : lcp) ans -= (ll)u;
70     return ans;
71 }
72 string lcsbstr(string s, string t) {
73     string r = s + '#' + t + '$';
74     int best = 0; int pos = 0;
75     vector<int> sa = suffix_array(r);
76     vector<int> lcp = lcp_construction(r, sa);
77     forr(i, 1, SZ(sa)) {
78         if(isInRange(sa[i - 1], 0, SZ(s)) &&
79             isInRange(sa[i], SZ(s) + 1, SZ(r) - 1))
80             {
81                 if(lcp[i-1] > best) best = lcp[i-1],
82                     pos = sa[i];
83             }
84         if(isInRange(sa[i], 0, SZ(s)) && isInRange(
85             sa[i - 1], SZ(s) + 1, SZ(r) - 1)) {
86             if(lcp[i-1] > best) best = lcp[i-1],
87                 pos = sa[i];
88         }
89     }
90     return r.substr(pos,best);
91 }

```

9.7 Suffix Automaton

```

1  struct state {
2      int len, link;

```

```

3     map<char,int> next;
4 };
5 const int MAXN = 300010;
6 state st[MAXN*2];
7 int sz, last;
8 void sa_init(){
9     forn(i,2*MAXN) {
10         st[i].next.clear();
11         st[i].link = 0;
12         st[i].len = 0;
13     }
14     last = st[0].len=0; sz=1;
15     st[0].link=-1;
16 }
17 void sa_extend(char c) {
18     int cur = sz++;
19     st[cur].len = st[last].len + 1;
20     int p;
21     for(p = last; p != -1 && !st[p].next.count(c);
22         p = st[p].link) {
23         st[p].next[c] = cur;
24     }
25     if (p == -1) st[cur].link = 0;
26     else {
27         int q = st[p].next[c];
28         if(st[p].len + 1 == st[q].len) st[cur].link
29             = q;
30         else {
31             int clone = sz++;
32             st[clone].len = st[p].len + 1;
33             st[clone].next = st[q].next;
34             st[clone].link = st[q].link;
35             while (p != -1 && st[p].next[c] == q) {
36                 st[p].next[c] = clone;
37                 p = st[p].link;
38             }
39             st[q].link = st[cur].link = clone;
40         }
41     }
42     last = cur;
43 }

```

9.8 Z Function

```

1 vector<int> z_function(string s) {
2     int n = (int) s.size();
3     vector<int> z(n);
4     for (int i = 1, l = 0, r = 0; i < n; ++i) {
5         if (i <= r)
6             z[i] = min(r - i + 1, z[i - l]);
7         while (i + z[i] < n && s[z[i]] == s[i + z[i]
8             ])
9             z[i]++;
10        if (i + z[i] - 1 > r)
11            l = i, r = i + z[i] - 1;
12    }
13    return z;
14 }

```

10 Tree

10.1 Centroid Cses

```

1 vector<int> g[MAXN];int n;
2 bool tk[MAXN];

```

```

3 int fat[MAXN]; // father in centroid decomposition
4 int szt[MAXN]; // size of subtree
5 int calcsz(int x, int f){
6     szt[x]=1;
7     for(auto y:g[x])if(y!=f&&!tk[y])szt[x]+=calcsz(
8         y,x);
9     return szt[x];
10 }
11 void cdfs(int x=0, int f=-1, int sz=-1){ // O(nlogn)
12     if(szt[x]<0)sz=calcsz(x,-1);
13     for(auto y:g[x])if(!tk[y]&&szt[y]*2>=sz){
14         szt[x]=0;cdfs(y,f,sz);return;
15     }
16     tk[x]=true;fat[x]=f;
17     for(auto y:g[x])if(!tk[y])cdfs(y,x);
18 }
19 void centroid(){memset(tk,false,sizeof(tk));cdfs()
20 ;}

```

10.2 Hld

```

1 struct hld {
2     #define rs(x) resize(x)
3     int n;
4     vector <vector<int>> g;
5     vector <tipo> values, sz, heavy, parent, depth,
6         top, r, pos;
7     segtree st; // iterative is faster
8     hld(int _n, vector <vector<int>> _g, vector <tipo>
9         > _v) {
10         n = _n; g = _g; values = _v;
11         sz.rs(n); parent.rs(n); depth.rs(n); top.rs(n);
12         pos.rs(n);
13         heavy.rs(n);
14         forn(i,n) heavy[i] = i;
15         dfs(0);
16         get_order(0,0);
17         st.build(r,r.size());
18     }
19     ll dfs(int v, int p = -1, int d = 0) {
20         depth[v] = d; parent[v] = p;
21         for(int &u : g[v]) {
22             if(u == p) continue;
23             ll tam = dfs(u,v,d+1);
24             // move heavy node to the beginning
25             if(szt[u] > sz[heavy[v]]) heavy[v] = u, swap(u
26                 ,g[v][0]);
27             sz[v] += tam;
28         }
29         sz[v] += 1;
30         return sz[v];
31     }
32     void get_order(int v, int t, int p = -1) { //
33         first child is heavy
34         pos[v] = r.size(); top[v] = t; r.pb(values[v]);
35         for(int u : g[v]) {
36             if(u == p) continue;
37             if(u == heavy[v]) get_order(u,t,v);
38             else get_order(u,u,v);
39         }
40     }
41     void op(ll &res, int a, int b) { // Set operation
42         res = max(res, st.query(pos[a],pos[b]));
43         // ----- for segtree recursive use -----
44     }
45 }

```

```

39 // res = max(res, st.query(pos[a],pos[b]).ans);
40 }
41 tipo query_hld(int a, int b) {
42     tipo res = NEUT;
43     for(; top[a] != top[b]; b = parent[top[b]]) {
44         if(depth[top[a]] > depth[top[b]]) swap(a, b);
45         op(res,top[b],b);
46     }
47     if(depth[a] > depth[b]) swap(a, b);
48     op(res,a,b);
49     return res;
50 }
51 void update_hld(int p, tipo val) { st.update(pos[
52     p],val); }
};

```

10.3 Lca

```

1 const int MAXN = 200005, LOG = 20;
2 struct LCA {
3     int n, root;
4     vector<vector<int>> g;
5     int jmp[MAXN][LOG], depth[MAXN];
6     void lca_dfs(int x) {
7         for(int u : g[x]) {
8             if(u == jmp[x][0]) continue;
9             jmp[u][0] = x; depth[u] = depth[x]+1;
10            lca_dfs(u);
11        }
12    }
13    LCA(int tam, vector<vector<int>> &tree, int r):
14        n(tam),root(r),g(tree) {
15        depth[root] = 0;
16        memset(jmp,-1,sizeof(jmp)); jmp[root][0] =
17            root;
18        lca_dfs(root);
19        forr(k, 1, LOG){ forn(i, n){
20            if(jmp[i][k-1]<0) jmp[i][k] = -1;
21            else jmp[i][k] = jmp[jmp[i][k-1]][k-1];
22        }
23    }
24    int lca(int x, int y){
25        if(depth[x] < depth[y]) swap(x,y);
26        for(int i = LOG-1; i >= 0; i--) {
27            if(depth[x]-(1<<i) >= depth[y]) x = jmp
28                [x][i];
29        }
30        if(x == y) return x;
31        for(int i = LOG-1; i >= 0; i--) {
32            if(jmp[x][i] != jmp[y][i]) x = jmp[x][i]
33                , y = jmp[y][i];
34        }
35        return jmp[x][0];
36    }
37    int dist(int x, int y) {
38        return depth[x] + depth[y] - 2 * depth[lca(
39            x,y)];
40    }
41 };

```

10.4 Sm To Large

```

1 struct query { // v: vrtx, c: col, idx_ans
2     ll v, c, idx;

```

```

3 };
4 vi g[MAXN];
5 vector<query> q[MAXN]; // Queries to answer
6 vi ans(MAXN, -1), color; // Answer to each query
7 unordered_map<ll, ll> cnt[MAXN];
8 int merge(int v, int u){
9     if(SZ(cnt[v]) < SZ(cnt[u])) swap(u, v);
10    for(auto [x, y]: cnt[u]){
11        cnt[v][x] += y;
12    }
13    cnt[u].clear();
14    return v;
15 }
16 void process_queries(int v, int v_repr){
17     for(auto &[_v, c, i]: q[v])
18         ans[i] = SZ(cnt[v_repr]);
19 }
20 int dfs(int v, int p){
21     int v_repr = v;
22     cnt[v][color[v]]++;
23     for(auto u: g[v]){
24         if(u == p) continue;
25         int u_repr = dfs(u, v);
26         v_repr = merge(v_repr, u_repr);
27     }
28     process_queries(v, v_repr);
29     return v_repr;
30 }
31 void solve(int m) {
32     dfs(0,-1);
33     forn(i,m) cout << ans[i] << " ";
34     cout << "\n";
35 }

```